Future University in Egypt

Faculty of Computers & Information Technology

Department of Computer Science



Wareeny

An AI-Powered Mobile Assistive System for Visually Impaired Users

Thesis Submitted in Partial Fulfillment of the Requirements for the Degree of Bachelor of Science in Computer Science

By:

Youssef Amr	20221949
Mohamed Moataz	20223394
Omar Ahmed Said	20225281
Ahmed Mohamed	20223031
Mohamed Wael	20223960
Mohamed Khaled	20224727

Under the Supervision of:

Professor: Prof. Abeer El Korany

© FUE 2025–2026

Abstract

This thesis presents Wareeny, an AI-assisted mobile application intended to support blind and low-vision users through camera-based perception, spoken feedback, and accessible interaction patterns. In its current implemented form, Wareeny is a Flutter-based mobile system organized around a mode-driven interface that includes home, navigation, stuff-finder, scan, read, and settings flows. The reading and scanning modules combine still-image capture with a routed vision-language pipeline, while the navigation and object-finding modules rely on on-device YOLOv8-based streaming detection. Currency reading is exposed as a dedicated task within the vision-language pipeline, and the architecture is explicitly designed to accept a specialized on-device Egyptian-pound banknote detector as a drop-in provider in future work. The application also provides bilingual English–Arabic interaction, text-to-speech output, haptic feedback, an active background emergency-and-safety subsystem, and persisted user preferences for language, speech rate, and haptics. Architecturally, the system separates presentation, feature controllers, shared AI services, backend integration, and lightweight persistence, producing a codebase that is both assistive in purpose and extensible in structure. The implemented repository therefore represents a credible functional foundation for an accessibility-oriented mobile assistant, while a dedicated offline banknote detector, broader Arabic-domain evaluation, formal accuracy benchmarking, and large-scale user studies remain future work.

Contents

Abstract **i**

List of Abbreviations **viii**

1 Introduction **2**

1.1 Background of the Study . . . 2
1.2 Problem Statement . . . 2
1.3 Aim and Objectives . . . 3
1.3.1 General Aim . . . 3
1.3.2 Specific Objectives . . . 3
1.4 Scope of the Project . . . 3
1.4.1 Included Features . . . 3
1.4.2 Excluded Features . . . 3
1.4.3 Constraints . . . 4

1.4.3.1 Data Constraints . . . 4
1.4.3.2 Hardware Constraints . . . 4
1.4.3.3 Time Constraints . . . 4
1.5 Significance of the Project . . . 4
1.5.1 Beneficiaries . . . 4
1.5.2 Potential Impact . . . 4
1.6 Project Methodology . . . 5
1.6.1 Development Approach . . . 5
1.6.2 Key Technologies and Tools . . . 5
1.7 Project Organization . . . 5

2 Literature Review **7**
2.1 Introduction . . . 7
2.2 Review of Related Systems and Studies . . . 7
2.2.1 Analysis of Commercial Assistive Applications . . . 7
2.2.1.1 Microsoft Seeing AI (2017) . . . 8
2.2.1.2 Google Lookout (2019) . . . 8
2.2.1.3 Envision AI (2018) . . . 8
2.2.1.4 E3rafli by Vodafone (2017) . . . 8
2.2.1.5 Comparative Analysis . . . 9
2.2.2 Review of Academic Research . . . 9
2.2.2.1 Image Quality and Interactive Visual Question Answering (VQA) . . . 9
2.2.2.2 Arabic Optical Character Recognition (OCR) . . . 10
2.2.2.3 Localized Currency Recognition . . . 10
2.2.2.4 Real-Time Obstacle Detection (Safety) . . . 10
2.2.2.5 Indoor Navigation and Structural Recognition . . . 11
2.2.2.6 Privacy-Preserving Computer Vision . . . 11
2.3 Research Gap and Summary . . . 12
2.3.1 The Geo-Linguistic Disparity Gap . . . 12
2.3.2 The Obsolescence of Legacy Computer Vision Architectures . . . 12
2.3.3 The Privacy-Utility Paradox . . . 13
2.3.4 Lack of Interactive Contextual Awareness . . . 13
2.4 Summary of Existing Technologies . . . 14
2.4.1 Cross-Platform Mobile Development Frameworks . . . 14
2.4.2 Real-Time Object Detection Architectures . . . 15
2.4.3 Large Multimodal Models for Semantic Understanding . . . 15
2.4.4 On-Device Optical Character Recognition Engines . . . 16
2.4.5 Hybrid Cloud-Local Processing Architectures . . . 16

3 System Analysis and Design **18**
3.1 System Overview and Proposed Concept . . . 18
3.2 Functional Requirements . . . 18
3.2.1 Indoor Assistance . . . 18
3.2.2 Outdoor Assistance . . . 19
3.2.3 OCR Features . . . 19
3.2.4 Audio and Haptics Service . . . 19
3.3 Non-Functional Requirements . . . 19

3.4 System Environment 20
3.4.1 Hardware Environment 20
3.4.2 Software Environment 21
3.5 System Architecture 22
3.6 Class Diagram 23
3.7 Use Case Diagram 24
3.8 Sequence Diagrams 26
3.8.1 General Flow Diagrams 26
3.8.2 Indoor Features 28
3.8.3 Outdoor Features 30
3.8.4 OCR Features 34
3.8.5 Safety and Emergency Features 35
3.8.6 Audio and Haptics Features 37
3.9 Entity-Relationship Diagram 38

4 System Design 40
4.1 Introduction 40
4.1.1 Design Orientation 40
4.1.2 Implemented Scope 40
4.2 Development Tools and Technologies 41
4.2.1 Flutter and Dart 41
4.2.2 Visual Input and Camera Capture 41
4.2.3 Vision-Language Reading Stack 41
4.2.4 Real-Time Object Detection Stack 41
4.2.5 Localization, Speech, and Preference Persistence 42
4.3 Architectural Evolution: Detection-and-OCR versus a Vision-Language Pipeline 42
4.3.1 Motivation for the Pivot 42
4.3.2 Comparative Justification 43
4.4 System Architecture 44
4.4.1 Presentation Layer 44
4.4.2 Application Control Layer 44
4.4.3 AI Service Layer 44
4.4.4 Backend Integration Layer 45
4.4.5 Persistence Layer 45
4.5 Application Map 45
4.5.1 Startup and Home Flow 45
4.5.2 Navigation Mode Flow 45
4.5.3 Stuff Finder Mode Flow 45
4.5.4 Scan Mode Flow 46
4.5.5 Read Mode Flow 46
4.5.6 Settings Flow 46
4.5.7 Emergency and Safety Flow 46
4.6 UI/UX Design 46
4.6.1 Accessibility Interaction Model 46
4.6.2 Visual Design Language 47
4.6.3 Screen Designs 47
4.6.4 Bilingual Experience 47

4.6.5 Feedback Strategy . . . 48
4.6.6 Safety UI Overlay and Onboarding . . . 48
4.7 Data Persistence . . . 48
4.7.1 Persistence Philosophy . . . 48
4.7.2 Persisted Data Model . . . 49
4.7.3 Emergency Contact Lifecycle . . . 50
4.7.4 Data That Is Not Persisted . . . 50

5 System Implementation and Testing 53
5.1 Introduction . . . 53
5.1.1 Implementation Approach . . . 53
5.2 System Tools . . . 53
5.2.1 Frontend and UI Tooling . . . 53
5.2.2 AI and Model Tooling . . . 54
5.2.3 Platform Services and Assets . . . 54
5.3 Core Features Implemented . . . 54
5.3.1 Mode-Based Navigation Shell . . . 55
5.3.2 Scan Mode with Hybrid VLM Routing . . . 55
5.3.3 Read Mode with Hybrid VLM Routing . . . 55
5.3.4 Real-Time Navigation Mode . . . 55
5.3.5 Stuff Finder Mode . . . 55
5.3.6 Settings, Localization, and Personalization . . . 56
5.3.7 Audio and Haptic Delivery . . . 56
5.3.8 Safety and Emergency Dispatch (CRUD on Emergency Contacts) . . . 56
5.4 User Interface Organized and Documented . . . 56
5.4.1 Reusable Widgets . . . 56
5.4.2 Screen Composition and Overlays . . . 56
5.4.3 Accessibility Semantics . . . 57
5.4.4 Documented Screens . . . 57
5.5 Source Code Organized and Documented . . . 57
5.5.1 Bootstrap and Dependency Injection . . . 57
5.5.2 Feature Controllers . . . 57
5.5.3 Shared AI and Detection Services . . . 58
5.5.4 Emergency Subsystem Classes . . . 58
5.5.5 Representative Source Listings . . . 59
5.6 Test Plan and Test Cases . . . 60
5.6.1 Functional Test Cases . . . 60
5.6.2 Integration and Fallback Cases . . . 61
5.6.3 Automated Test Coverage . . . 61
5.7 Testing the Core Features with Sample Data . . . 61
5.7.1 Reading and Scanning Samples . . . 61
5.7.2 Navigation Samples . . . 62
5.7.3 Stuff-Finder Samples . . . 62
5.7.4 Settings Persistence Samples . . . 62
5.7.5 Emergency Dispatch Verification . . . 62
5.7.6 Testing Summary and Interpretation . . . 63
5.8 Comparative Capability Evaluation . . . 63

6 Conclusions and Future Work **66**
6.1 Introduction . . . 66
6.2 Project Summary . . . 66
6.2.1 Functional Summary . . . 66
6.2.2 Architectural Summary . . . 66
6.3 Project Contributions . . . 67
6.3.1 Accessibility-Centred Interaction Model . . . 67
6.3.2 Hybrid, Tiered Understanding Architecture . . . 67
6.3.3 Real-Time Local Detection Integration . . . 67
6.3.4 Extensible Service Structure . . . 67
6.4 Conclusion and Future Work . . . 67
6.4.1 Conclusion . . . 67
6.4.2 Near-Term Future Work . . . 67
6.4.3 Long-Term Future Work . . . 68

Bibliography **70**

List of Figures

3.1 System Architecture Diagram. 22
3.2 Class Diagram. 23
3.3 Use Case Diagram. 25
3.4 General Flow 1: OCR Processing. 26
3.5 General Flow 2: Object Detection Processing. 27
3.6 General Flow 3: Navigation Processing. 27
3.7 Sequence Diagram: Indoor Object Detection and Distance Estimation. 28
3.8 Sequence Diagram: In-Hand Object Detection. 29
3.9 Sequence Diagram: Brand and Currency Detection. 29
3.10 Sequence Diagram: Outdoor Object Detection. 30
3.11 Sequence Diagram: Outdoor Object Finder. 31
3.12 Sequence Diagram: Outdoor Reader for Signs. 31
3.13 Sequence Diagram: Outdoor Navigation. 32
3.14 Sequence Diagram: Traffic Light Status Detection. 33
3.15 Sequence Diagram: Read Document. 34
3.16 Sequence Diagram: QA Mode (Visual Query). 34
3.17 Sequence Diagram: Danger Word Alerts. 35
3.18 Sequence Diagram: Danger Mode (Hazard Detection). 35
3.19 Sequence Diagram: Emergency Auto Call and Location Sharing. 36
3.20 Sequence Diagram: Privacy Mode (Secure Shutdown). 36
3.21 Sequence Diagram: Directional Audio. 37
3.22 Entity-Relationship Diagram. 38

4.1 Home and Scan entry surface with the camera preview, mode carousel, and top status bar. 47
4.2 Navigation mode showing live detection overlays and directional status indicators. 48
4.3 Settings mode with language, speech-rate, and haptics configuration cards. 49

5.1 Scan mode after a capture, showing the spoken-result overlay and repeat action. 57
5.2 Read mode with stop, repeat, skip, and cancel controls during progressive speech. 58
5.3 Stuff Finder target picker merging custom assistive classes with COCO classes. 59

List of Tables

2.1 Comparison of Commercial Assistive Applications 9

3.1 Hardware Environment Specifications 20
3.2 Software Environment Technologies 21

4.1 Classical Detection/OCR versus the Vision-Language Pipeline for Accessibility 43
4.2 Numerical Performance Comparison: Classical Pipeline vs. Wareeny's Hybrid Approach 43
4.3 Complete On-Device Persistence Model (`SharedPreferences`) 50
4.4 Data Categories Intentionally Not Persisted 51

5.1 Implementation Tools and Versions 54
5.2 Capability Comparison of Assistive Applications 63
5.3 Numerical Performance Comparison Across Key Metrics 64

List of Abbreviations

AI	Artificial Intelligence
API	Application Programming Interface
COCO	Common Objects in Context
CPU	Central Processing Unit
CV	Computer Vision
DI	Dependency Injection
EGP	Egyptian Pound
FFI	Foreign Function Interface
GAN	Generative Adversarial Network
GPU	Graphics Processing Unit
LF-VQA	Long-Form Visual Question Answering
LMM	Large Multimodal Models
MENA	Middle East and North Africa
OCR	Optical Character Recognition
RAM	Random Access Memory
SSE	Server-Sent Events
TTS	Text-to-Speech
VLM	Vision-Language Model
VQA	Visual Question Answering
WHO	World Health Organization

YOLO You Only Look Once

1

Chapter One

Introduction

Chapter 1

Introduction

This chapter introduces the project, provides background context, defines the problem, and outlines the objectives and significance of the Wareeny assistive system for visually impaired users.

1.1 Background of the Study

This project focuses on assistive technology for blind and low-vision users using computer vision, Optical Character Recognition (OCR), audio, and haptic feedback. Smartphones now include powerful cameras, sensors, and on-device AI capabilities. These capabilities create an opportunity to deliver real-time environmental awareness without specialized hardware.

Vision-impaired users face daily barriers both indoors and outdoors. Objects, obstacles, text, and signals are often inaccessible without assistance. Human assistance is not always available, and traditional aids do not provide contextual understanding.

Current assistive applications handle isolated tasks. Many focus only on OCR or basic object detection. They lack context awareness, struggle in cluttered environments, and often overwhelm users with unnecessary information. There is a gap in integrated systems that combine object detection, text understanding, safety alerts, and directional feedback. Few systems filter information based on user intent or risk, and privacy handling remains limited.

This study addresses the need for a unified mobile system that delivers accurate, timely, and safe assistance tailored to real-world situations. By leveraging modern deep learning architectures and mobile computing capabilities, the proposed system aims to significantly improve independence and safety for visually impaired individuals.

1.2 Problem Statement

Blind and low-vision users lack a single reliable mobile system that provides real-time contextual understanding of their surroundings. Existing solutions handle isolated functions and fail in complex environments. They do not prioritize safety-critical information and offer limited privacy control. This results in reduced independence and increased risk during daily activities.

1.3 Aim and Objectives

1.3.1 General Aim

The general aim of this project is to build a mobile assistive system that supports blind and low-vision users through real-time vision, audio, and haptic feedback. The system seeks to improve safety, independence, and access to information in both indoor and outdoor spaces.

1.3.2 Specific Objectives

The specific objectives of this project are:

1. Detect indoor objects and estimate distance with high accuracy.
2. Detect the object held in the user's hand with low false detection rate.
3. Read currency and identify value quickly.
4. Detect outdoor obstacles and traffic lights with real-time response with very low latency.
5. Read text and answer user questions from documents with correct intent recognition.
6. Trigger danger mode and emergency call very quickly.

1.4 Scope of the Project

1.4.1 Included Features

The following features are included in the scope of this project:

- Indoor object detection, brand recognition, and in-hand detection.
- Outdoor object detection, obstacle awareness, and object finding.
- OCR for documents, signs, and labels.
- Integrating LMM-based question answering.
- Audio cues and haptic alerts.
- Emergency call and danger mode activation.

1.4.2 Excluded Features

The following aspects are excluded from the scope of this project:

- **Hardware Manufacturing:** The project focuses on a software-based mobile application. It does not include the development or sale of custom wearable hardware (such as smart glasses).

1.4.3 Constraints

The project is subject to the following constraints:

1.4.3.1 Data Constraints

- Limited availability of labeled datasets for in-hand detection, brand recognition, and currency.
- Dataset bias toward common objects and environments.
- Language and currency coverage restricted to selected regions.

1.4.3.2 Hardware Constraints

- Mobile phone CPU, GPU, and memory limitations.
- Camera resolution and low-light performance vary across devices.
- Battery consumption restricts continuous real-time processing.

1.4.3.3 Time Constraints

- Fixed academic timeline limits model training and optimization cycles.
- Restricted time for large-scale user testing.
- Limited duration for dataset collection and annotation.

1.5 Significance of the Project

1.5.1 Beneficiaries

The project benefits multiple stakeholders:

- **Blind and low-vision users:** Gain independence in daily activities.
- **Caregivers:** Gain safety assurance for their dependents.
- **Researchers:** Gain access to a modular assistive vision system for further study.
- **Organizations:** Gain a deployable accessibility tool for their constituents.

1.5.2 Potential Impact

The potential impact of this project includes:

- Reduces daily dependency on human assistance.
- Combines vision, language, audio, and haptics in one integrated system.
- Focuses on context filtering to reduce cognitive noise.

- Helps blind individuals perform daily tasks with greater ease and confidence.

1.6 Project Methodology

1.6.1 Development Approach

The project follows an iterative Agile development approach with the following characteristics:

- Incremental feature delivery with regular milestones.
- Continuous testing after each module integration.
- User-centered design with feedback-driven refinement.

1.6.2 Key Technologies and Tools

The following technologies and tools are employed in the development:

- Deep learning models for object detection and OCR.
- Mobile frameworks for on-device inference.
- Text recognition and assistant LMM models.
- Audio output and vibration APIs for feedback.
- Smartphone camera, microphone, and motion sensors.

1.7 Project Organization

This thesis is organized as follows:

- Chapter One: Introduces the project, presents the problem statement, defines the aim and objectives, and outlines the scope.
- Chapter Two: Provides a literature review of assistive vision systems, object detection, OCR, and audio feedback technologies.
- Chapter Three: Presents the system analysis and design, including models, datasets, and design decisions.
- Chapter Four: Presents the design of the implemented application.
- Chapter Five: Describes the implementation and testing of the system.
- Chapter Six: Concludes the thesis and identifies future work.

2

Chapter Two

Literature Review

Chapter 2

Literature Review

2.1 Introduction

Visual impairment represents a significant global health challenge, affecting approximately 2.2 billion people worldwide, with over 1 billion cases that could have been prevented or remain unaddressed [19]. These individuals face substantial barriers in navigating their environments, accessing textual information, and maintaining independent daily activities. The burden of visual impairment falls disproportionately on developing regions, particularly in the Middle East and North Africa (MENA), where access to assistive technologies remains limited.

Recent advances in artificial intelligence and edge computing have opened new possibilities for developing sophisticated assistive technologies. However, the market remains dominated by solutions designed with a Western-centric focus, lacking robust support for non-Latin scripts, regional entities, and cultural contexts. In Egypt and the broader Arab world, visually impaired individuals encounter a critical scarcity of assistive technologies that are linguistically adapted for Arabic and culturally tailored to recognize region-specific currency, products, and environmental elements. Furthermore, the majority of existing commercial solutions rely heavily on cloud-based processing architectures, raising significant privacy concerns regarding the transmission of sensitive personal data, including images of users' environments and biometric information.

This project, Wareeny, proposes a novel, comprehensive mobile application designed to bridge these specific accessibility gaps. By integrating state-of-the-art 2025 AI architectures—combining real-time AI processing for safety with Large Multimodal Models (LMMs) for semantic understanding—Wareeny aims to empower visually impaired users in the Arab world with a reliable, localized, and privacy-preserving assistive tool.

2.2 Review of Related Systems and Studies

The domain of assistive technology for the visually impaired is characterized by a mix of commercial mobile applications and academic research targeting specific computer vision problems. A critical review of these systems highlights both the progress made and the significant limitations that persist.

2.2.1 Analysis of Commercial Assistive Applications

Several prominent mobile applications currently serve the visually impaired community. However, a detailed analysis reveals deficiencies in accuracy, localization, and usability, particularly for users in the MENA region.

2.2.1.1 Microsoft Seeing AI (2017)

Microsoft Seeing AI [7], launched in 2017, represents one of the earliest comprehensive AI-powered assistive applications, providing multiple channels for text reading, scene description, and object identification. The application leverages cloud-based computer vision models to deliver real-time audio feedback through smartphone cameras. However, systematic user studies have revealed several critical limitations. First, the system exhibits reduced accuracy in complex, cluttered environments, where it struggles to distinguish foreground objects from background elements. Second, users report experiencing cognitive overload due to the system's tendency to narrate non-essential environmental details rather than prioritizing safety-critical information such as obstacles or hazards. Third, its natural language processing capabilities are optimized predominantly for English, with substantially degraded performance when processing Arabic text due to the morphological complexity and cursive nature of Arabic script. Finally, the application's exclusive availability on iOS limits accessibility for Android users, particularly in developing regions where Android devices dominate the market.

2.2.1.2 Google Lookout (2019)

Google Lookout [8], released in 2019, employs camera-based computer vision to identify objects, text, and people in the user's environment. The application offers three operational modes: Explore, Shopping, and Scan, each optimized for specific use cases. Despite these capabilities, the system faces multiple technical and accessibility limitations. First, it is exclusively available on the Android platform, effectively excluding the substantial iOS user base. Second, users have reported system instability characterized by frequent application crashes, particularly during extended usage sessions or when processing complex visual scenes. Third, the continuous camera streaming and real-time AI inference impose substantial computational demands, resulting in rapid battery depletion that limits practical usage duration. Most critically, the application relies predominantly on cloud-based processing, requiring transmission of captured images to Google's servers for analysis. This architecture introduces privacy vulnerabilities, particularly when users scan sensitive personal documents, financial information, or private correspondence. Additionally, the cloud dependency introduces latency in response times and renders the application non-functional in offline scenarios or areas with limited network connectivity.

2.2.1.3 Envision AI (2018)

Envision AI [9], introduced in 2018, has gained recognition for its optical character recognition (OCR) capabilities and cross-platform availability on both iOS and Android devices. The application provides functionality for document reading, object recognition, scene description, and color identification. However, it exhibits significant limitations in regional localization and technical architecture. The system's object detection models are trained primarily on Western-centric datasets such as COCO [11], resulting in poor recognition accuracy for region-specific items including local Egyptian products, Middle Eastern food items, and Arabic-labeled packaging. Currency recognition is similarly limited, failing to accurately identify Egyptian pounds or other regional currencies, particularly when banknotes are worn, folded, or partially visible. From a technical perspective, the application relies on computer vision architectures predating 2020, which lack the inference speed and semantic understanding capabilities of modern state-of-the-art models such as YOLOv8 and vision transformers. This results in higher latency and reduced accuracy compared to contemporary deep learning approaches. Furthermore, while the application offers a hybrid processing model combining on-device and cloud-based inference, the lack of complete offline functionality limits its applicability in connectivity-constrained environments.

2.2.1.4 E3rafli by Vodafone (2017)

E3rafli [10], developed by Vodafone Egypt in 2017, represents one of the few assistive applications specifically designed for the Egyptian market and Arabic-speaking users. The application provides a native Arabic user interface

and voice feedback system, addressing a critical linguistic gap in the assistive technology landscape. However, its technical capabilities remain substantially limited compared to global alternatives. The application demonstrates acceptable performance only in two specific domains: color detection and Egyptian currency recognition. These features employ relatively simple computer vision algorithms that can operate effectively on limited training data. In contrast, the application's general object detection capabilities suffer from low accuracy, likely due to insufficient training data and the use of lightweight, less sophisticated detection models. Scene understanding functionality is similarly restricted, providing only basic environmental descriptions without contextual awareness or safety-relevant information filtering. Document reading and OCR performance are particularly weak, with poor text recognition accuracy for both printed and handwritten Arabic text, limiting the application's utility for accessing written information. The application is available exclusively on Android devices and employs primarily on-device processing, offering privacy advantages over cloud-dependent alternatives. However, the limited feature set and accuracy constraints render it insufficient as a comprehensive assistive tool for daily independence, necessitating supplementation with additional assistive technologies.

2.2.1.5 Comparative Analysis

Table 2.1 provides a comparative analysis of the key commercial assistive applications, highlighting their strengths and limitations across critical dimensions.

Table 2.1: Comparison of Commercial Assistive Applications

Application	Year	Platform	Arabic Support	Processing	Key Limitations
Microsoft Seeing AI [7]	2017	iOS only	Limited	Cloud-based	Inaccurate in clutter; cognitive overload
Google Lookout [8]	2019	Android only	Limited	Cloud-based	Frequent crashes; high battery usage; privacy concerns
Envision AI [9]	2018	iOS & Android	Weak	Hybrid	Poor localization; outdated CV models
E3rafli (Vodafone) [10]	2017	Android only	Native	Local	Limited features; inaccurate object detection

2.2.2 Review of Academic Research

While commercial applications reveal practical limitations in deployment, academic research provides the theoretical foundation and technical innovations necessary to address these challenges. Recent scholarly work has concentrated on three critical dimensions: (1) optimizing state-of-the-art artificial intelligence models for resource-constrained mobile deployment, (2) developing localization techniques for non-Western languages and cultural contexts, and (3) implementing privacy-preserving architectures that eliminate dependence on cloud infrastructure. This section systematically reviews key contributions in each domain, highlighting their relevance to the development of next-generation assistive systems.

2.2.2.1 Image Quality and Interactive Visual Question Answering (VQA)

A persistent challenge in assistive computer vision is that visually impaired users frequently capture images that are blurry, poorly framed, or partially obscured, resulting from the inherent difficulty of camera alignment without

to address the specific informational needs of users in these suboptimal capture scenarios. Huh et al. (2024) conducted a comprehensive user study to address this limitation, proposing a "Long-Form Visual Question Answering" (LF-VQA) framework [1]. Their empirical findings demonstrate that visually impaired users derive significantly greater value from interactive multimodal models that support iterative questioning, allowing them to request clarification, probe specific regions of interest, and obtain contextual information beyond initial descriptions. The study revealed that interactive dialogue-based systems improved task completion rates by 40% compared to static captioning approaches, particularly for scenarios involving document analysis, navigation decisions, and object identification. This research validates the critical necessity of transitioning from one-way image description to bidirectional conversational assistive agents capable of natural language interaction.

2.2.2.2 Arabic Optical Character Recognition (OCR)

Optical character recognition systems, while achieving accuracy rates exceeding 95% on printed Latin scripts, exhibit substantially degraded performance when processing Arabic text due to several linguistic and typographic characteristics. Arabic script is inherently cursive, with character shapes varying significantly based on positional context (isolated, initial, medial, or final forms). Additionally, diacritical marks, ligatures, and right-to-left text flow introduce complexities that generic multilingual OCR engines struggle to handle accurately. Ayadi et al. (2025) conducted a systematic investigation of this challenge, proposing specialized deep learning architectures including optimized Generative Adversarial Networks (GANs) specifically trained on Arabic handwritten and printed text datasets [2]. Their experimental results demonstrated that generic multilingual OCR models (such as Tesseract with Arabic language packs) achieve only 68% character-level accuracy on handwritten Arabic text, whereas their specialized GAN-based architecture achieved 89% accuracy on the same test set. The study conclusively establishes that reliable Arabic OCR for assistive applications requires dedicated, culturally-aware models trained on region-specific datasets encompassing diverse Arabic fonts, handwriting styles, and document types. This finding has direct implications for developing assistive systems targeting Arabic-speaking populations in the MENA region.

2.2.2.3 Localized Currency Recognition

Financial independence represents a critical component of daily autonomy for visually impaired individuals, requiring reliable identification of local currency denominations. However, global currency recognition models exhibit poor generalization to regional currencies, particularly for nations outside North America and Western Europe. This limitation is exacerbated when banknotes are worn, folded, partially visible, or photographed under variable lighting conditions. Ghanem et al. (2025) addressed this gap by developing a specialized deep learning system for Egyptian currency recognition, constructing a custom dataset of 5,000 images encompassing all current Egyptian pound denominations across varying conditions [3]. Their comparative evaluation demonstrated that transfer learning from global currency datasets achieved only 72% recognition accuracy on Egyptian notes, whereas their locally-trained convolutional neural network architecture achieved 94% accuracy under controlled conditions and 87% accuracy with worn or partially occluded banknotes. The study provides empirical validation for the necessity of region-specific training data and localized model development in assistive applications, particularly for underrepresented currencies and cultural contexts. These findings directly inform the design requirements for currency recognition modules in assistive systems targeting the Egyptian and broader MENA market, and they directly motivate Wareeny's dedicated on-device Egyptian-pound banknote detector described in Chapters 4 and 5.

2.2.2.4 Real-Time Obstacle Detection (Safety)

For outdoor navigation assistance, detection latency directly impacts user safety, as delayed obstacle warnings provide insufficient reaction time to prevent collisions. Real-time object detection on mobile devices thus represents a critical technical requirement, necessitating inference speeds exceeding 10 frames per second while maintaining

(You Only Look Once) architecture specifically optimized for detecting outdoor navigation obstacles including curbstones, poles, stairs, and pedestrian barriers [4]. Their architecture modifications included lightweight feature extractors, optimized anchor configurations for obstacle aspect ratios, and pruned network layers to reduce computational complexity. Experimental evaluation on mobile devices demonstrated that YOLO-OD achieved 15.2 FPS inference speed with 88.3% mean Average Precision (mAP) for obstacle detection, compared to 8.7 FPS and 91.2% mAP for standard YOLOv5. The research conclusively demonstrates that single-stage detection architectures offer the optimal trade-off between inference speed and accuracy for real-time mobile deployment, providing sufficient performance for safety-critical navigation assistance while operating entirely on-device without cloud dependency. This work validates the technical feasibility of deploying state-of-the-art object detection models on resource-constrained mobile hardware for latency-sensitive assistive applications.

2.2.2.5 Indoor Navigation and Structural Recognition

Indoor navigation for visually impaired users requires identification of specific structural and architectural elements beyond generic object classes. Critical indoor elements include doorways, staircases, elevators, handrails, chairs, and tables, each requiring precise localization for safe navigation and spatial awareness. Wojtowicz (2025) conducted a comprehensive comparative evaluation of YOLOv8 and YOLOv9 architectures within assistive Mixed Reality (MR) applications, specifically assessing their performance on indoor navigation-relevant object classes [5]. The experimental study employed a custom indoor navigation dataset comprising 12,000 annotated images across diverse indoor environments including offices, homes, and public buildings. Results demonstrated that YOLOv8 achieved 91.7% precision and 87.3% recall for detecting doors, 89.2% precision for staircases, and 85.6% precision for identifying empty seats. Importantly, YOLOv8 maintained real-time inference speeds of 12.8 FPS on mid-range mobile devices, meeting the responsiveness requirements for assistive navigation applications. The study's findings validate YOLOv8's suitability for the "Indoor Mode" of navigation assistance systems, confirming its ability to accurately detect structurally important elements while maintaining computational efficiency compatible with mobile deployment constraints.

2.2.2.6 Privacy-Preserving Computer Vision

The architectural reliance on cloud processing in existing commercial assistive applications creates substantial privacy vulnerabilities, as users must continuously transmit sensitive visual data including images of their home environments, personal documents, financial information, and biometric features to third-party servers. This data exposure contradicts fundamental privacy principles and creates risks of unauthorized access, data breaches, and secondary use of personal information. Martin & Moon (2025) proposed a comprehensive "Privacy-Preserving Image Captioning" framework that addresses these concerns through a hybrid architecture combining on-device detection with selective encryption [6]. Their approach employs a multi-stage pipeline: (1) lightweight object detection models run locally to identify sensitive visual elements including faces, text, and personally identifiable information; (2) detected sensitive regions are encrypted or redacted using partial image encryption techniques before any data transmission; (3) only privacy-filtered images are transmitted for cloud-based semantic analysis when needed. Experimental evaluation demonstrated that this approach reduces sensitive data exposure by 94% while maintaining 89% of the semantic understanding capabilities compared to unrestricted cloud processing. The framework supports implementation of an On-Device Privacy Guard architecture, ensuring that privacy-critical detection and filtering occurs locally before any network transmission, effectively providing user control over data exposure. This research validates the technical feasibility of privacy-preserving assistive systems that balance functional capability with data protection requirements.

2.3 Research Gap and Summary

A systematic analysis of both commercial applications and academic literature reveals a fragmented ecosystem in assistive technology for visually impaired users. While significant progress has been achieved in individual technical domains—including object detection, optical character recognition, and natural language processing—the field lacks integrated, culturally-aware, and privacy-centric systems that address the multidimensional needs of users in non-Western contexts. This section identifies four critical research gaps that current state-of-the-art solutions fail to address comprehensively.

2.3.1 The Geo-Linguistic Disparity Gap

Existing commercial assistive applications and academic research exhibit a pronounced Western-centric development bias, optimizing for English-language contexts and environments typical of North American and Western European settings. Global applications such as Microsoft Seeing AI [7] and Envision AI [9] demonstrate high performance on Latin scripts and Western product recognition but exhibit substantially degraded accuracy when deployed in Middle Eastern contexts. This geo-linguistic disparity manifests across multiple technical dimensions:

Linguistic Limitations: Current systems struggle fundamentally with the morphological complexity of Arabic script. The cursive, context-dependent nature of Arabic text, combined with diacritical marks and bidirectional text flow, results in OCR error rates 3–5 times higher than equivalent Latin-script recognition. Furthermore, Arabic dialect variation across the MENA region introduces additional recognition challenges that monolithic language models fail to address. Existing applications provide limited or non-existent support for Arabic voice commands, text-to-speech output with natural Arabic prosody, and culturally appropriate audio feedback mechanisms.

Cultural and Environmental Blindness: Object detection models trained on Western-centric datasets such as COCO [11] demonstrate poor generalization to region-specific entities. Critical daily-use items including local food products, Arabic-labeled packaging, regional clothing items, and Egyptian currency denominations remain unrecognized or misclassified. This limitation extends to architectural and environmental elements, as Middle Eastern building structures, street furniture, signage conventions, and urban layouts differ significantly from Western training environments, resulting in degraded navigation assistance accuracy.

Dataset Availability: The scarcity of publicly available, annotated datasets for Arabic text in natural scenes, Egyptian products, Middle Eastern architectural elements, and regional currency severely constrains the development of localized models. This data gap perpetuates the Western-centric bias and prevents effective transfer learning for MENA-specific applications.

2.3.2 The Obsolescence of Legacy Computer Vision Architectures

A substantial proportion of deployed assistive applications relies on computer vision models and architectural paradigms developed between 2017 and 2020, predating significant advances in efficient neural network design, transformer-based architectures, and mobile-optimized inference engines. This technological lag creates a critical performance-accuracy-efficiency trade-off that limits practical utility:

Latency and Real-Time Performance: Legacy cloud-dependent architectures introduce end-to-end latencies ranging from 800ms to 3 seconds, encompassing image capture, network transmission, cloud inference, and response delivery. For safety-critical navigation scenarios, this latency is unacceptable, as obstacles require detection and warning within 200–300ms to enable appropriate user response. Older on-device models, while offering lower latency, suffer from reduced accuracy due to over-aggressive model compression and outdated architectural

designs.

Semantic Understanding Limitations: Pre-2023 object detection models typically operate as simple "taggers," providing class labels and bounding boxes without contextual understanding, spatial relationships, or scene-level semantic reasoning. These systems cannot answer queries such as "Is this intersection safe to cross?" or "Which door leads to the exit?" without explicit rule-based programming. The absence of semantic understanding results in information overload, as systems cannot filter safety-relevant information from inconsequential environmental details.

Architectural Inefficiency: Modern architectures including YOLOv8/v10 [15], EfficientDet, and vision transformers offer 2–3× improvements in inference efficiency and 15–20% accuracy gains compared to legacy models, while requiring comparable or reduced computational resources. The gap in utilizing next-generation 2024–2025 architectures, including Large Multimodal Models (LMMs) [16] for semantic understanding, represents a missed opportunity for substantial capability enhancement in assistive applications.

2.3.3 The Privacy-Utility Paradox

Current assistive technology architectures present users with a forced dichotomy between functional capability and data privacy, requiring them to choose between high-accuracy cloud-based systems that expose sensitive personal information and privacy-preserving offline tools with substantially limited functionality. This paradox manifests across multiple dimensions:

Cloud Dependency and Data Exposure: Commercial applications achieving high accuracy rates (Microsoft Seeing AI, Google Lookout, Envision AI) require continuous streaming of visual data to cloud servers for inference. This architecture exposes users to multiple privacy risks: (1) transmission of images containing sensitive personal information, private documents, and home environment details to third-party commercial servers; (2) potential secondary use of collected data for model training, product development, or commercial purposes beyond the stated assistive function; (3) vulnerability to data breaches, unauthorized access, or government surveillance; and (4) lack of user control over data retention, deletion, and usage policies.

Limited Offline Capabilities: Existing privacy-preserving approaches that operate entirely offline (such as basic on-device OCR or simple color detection) offer substantially reduced functionality, with accuracy rates 20–40% lower than cloud-based alternatives. These systems typically support only individual isolated functions rather than integrated multimodal assistance, forcing users to employ multiple applications for comprehensive environmental understanding.

Research Gap in Privacy-First Architectures: Academic literature reveals a notable scarcity of research addressing privacy-preserving architectures that maintain competitive accuracy and comprehensive functionality. Specifically, there is a gap in implementing on-device privacy filtering and selective redaction systems that protect sensitive information while enabling cloud-assisted semantic understanding when explicitly authorized by users. Furthermore, insufficient research addresses the optimization of state-of-the-art models for complete on-device execution on mobile hardware, particularly for computationally intensive tasks such as scene understanding and visual question answering.

2.3.4 Lack of Interactive Contextual Awareness

Existing assistive applications predominantly operate as unidirectional communication systems, delivering pre-generated descriptions or simple object labels without supporting natural conversational interaction or context-

visually impaired users strongly prefer interactive Visual Question Answering (VQA) systems over static image captioning. However, current commercial applications fail to provide this capability due to several limitations:

Absence of Dialogue Systems: Most applications provide single-utterance outputs (e.g., "I see a chair, a table, and a person") without supporting follow-up queries such as "Where is the chair located?" or "Is the path to the door clear?" This limitation prevents users from obtaining specific information relevant to their immediate needs and decision-making contexts.

Lack of Intelligent Information Filtering: Existing systems employ indiscriminate description generation, narrating all detected objects regardless of relevance to user safety or current task context. This results in cognitive overload and difficulty extracting actionable information from verbose environmental descriptions. There is a critical need for context-aware filtering that prioritizes safety-critical information (obstacles, hazards, navigation landmarks) over incidental environmental details.

Limited Semantic Reasoning: Current applications cannot perform scene-level reasoning or spatial relationship understanding required for complex queries. Questions such as "Can I reach the exit without encountering obstacles?" or "What items are on the table within my reach?" remain beyond the capabilities of existing assistive systems.

In conclusion, while individual technical components for superior assistive systems exist in isolation within recent research contributions, no single integrated platform effectively combines localized Arabic AI, real-time safety-oriented processing, privacy-preserving architectures, and interactive contextual awareness into a unified mobile solution. Wareeny is proposed specifically to address this multi-dimensional research gap, providing a comprehensive assistive system tailored to the linguistic, cultural, and privacy requirements of visually impaired users in the Arab world.

2.4 Summary of Existing Technologies

To address the identified research gaps and develop a comprehensive assistive system for visually impaired users in the Arab world, this project strategically leverages a curated suite of state-of-the-art technologies, frameworks, and platforms available as of 2025. This section provides a systematic overview of the key technological enablers that facilitate the development of a responsive, localized, and privacy-aware mobile assistive application. The technology stack is selected based on three primary criteria: (1) technical maturity and proven performance in production environments, (2) compatibility with mobile deployment constraints including computational efficiency and battery consumption, and (3) support for privacy-preserving on-device processing architectures.

2.4.1 Cross-Platform Mobile Development Frameworks

Modern assistive applications face the critical requirement of broad platform accessibility, necessitating support for both iOS and Android ecosystems to maximize user reach across diverse socioeconomic contexts. Flutter [12], developed by Google and released in stable production form in 2018, has emerged as the leading cross-platform mobile development framework for assistive technology applications. Flutter employs the Dart programming language and a reactive framework architecture, rendering user interfaces through a custom graphics engine (Skia) that provides pixel-perfect consistency across platforms without reliance on platform-specific UI components. For assistive technology development, Flutter offers several critical technical advantages. First, it provides native performance characteristics with 60 FPS rendering capabilities, essential for real-time camera preview and visual feedback interfaces. Second, Flutter supports direct integration with platform-specific APIs through platform chan-

framework facilitates seamless embedding of machine learning inference engines including TensorFlow Lite [13] and PyTorch Mobile [14] through Foreign Function Interface (FFI) bindings, enabling on-device AI processing without network dependency. Finally, Flutter's plugin ecosystem includes pre-built packages for text-to-speech, speech recognition, and camera manipulation, accelerating development of assistive features while maintaining code reusability across platforms.

2.4.2 Real-Time Object Detection Architectures

Real-time object detection represents a foundational capability for assistive navigation and environmental awareness systems. The technical requirements are stringent: detection algorithms must achieve inference speeds exceeding 10 frames per second on mobile hardware while maintaining mean Average Precision (mAP) above 35% on standard benchmark datasets. The YOLO (You Only Look Once) family of single-stage detection architectures has established dominance in this domain due to their optimal balance of speed and accuracy. YOLOv8 [15], released by Ultralytics in 2023, represents the current state-of-the-art in efficient object detection. The architecture introduces several technical innovations over previous versions: (1) an anchor-free detection head that eliminates the need for predefined anchor box configurations, improving generalization across object scales; (2) a modified CSP-Darknet backbone with cross-stage partial connections that reduce computational complexity while maintaining feature extraction capability; (3) a Path Aggregation Network (PANet) feature pyramid that enhances multi-scale object detection; and (4) optimized loss functions combining Classification, Localization, and Distribution Focal Loss for improved training convergence. Empirical evaluation demonstrates that YOLOv8n (nano variant) achieves 37.3% mAP on the COCO dataset [11] while maintaining inference speeds of 15.2 FPS on mid-range mobile processors with only 6.2 MB model size. This performance profile makes it ideally suited for safety-critical assistive applications requiring detection of dynamic obstacles, traffic signals, indoor structural elements, and in-hand objects with minimal latency. The model's efficiency enables continuous real-time processing without excessive battery consumption, addressing a critical constraint identified in existing commercial applications such as Google Lookout [8]. In the implemented system, the YOLOv8n architecture is reused in three distinct roles—a general detector for navigation, a custom-trained detector for object finding, and a custom-trained Egyptian-pound banknote detector for money reading.

2.4.3 Large Multimodal Models for Semantic Understanding

The emergence of Large Multimodal Models (LMMs) in 2023–2024 has fundamentally transformed the landscape of visual understanding and human-computer interaction. Unlike traditional computer vision systems that operate as discriminative classifiers (identifying predefined object categories), LMMs employ generative transformer architectures trained on billions of image-text pairs, enabling semantic reasoning, contextual understanding, and natural language interaction with visual content. OpenAI's GPT-4 with Vision (GPT-4V) [16], released in September 2023, exemplifies this new paradigm. The model processes images through a vision encoder that generates visual token representations, which are then integrated with text tokens in a unified transformer architecture capable of joint reasoning across modalities. This approach enables several capabilities critical for assistive technology: (1) Visual Question Answering (VQA), allowing users to pose natural language queries about image content and receive contextually relevant responses; (2) scene understanding with spatial reasoning, enabling descriptions of object relationships, spatial layouts, and navigational affordances; (3) document understanding, including extraction and interpretation of text within complex layouts, tables, and forms; and (4) interactive dialogue, supporting multi-turn conversations that allow users to iteratively refine queries and obtain clarification. For assistive applications, LMMs address the fundamental limitation of static image captioning identified in research by Huh et al. (2024) [1], enabling visually impaired users to interrogate their environment through natural conversation. However, large proprietary LMMs operate exclusively via cloud APIs, introducing latency, cost, and privacy concerns.

vision-language model (the Gemma family) that can run on project-controlled infrastructure and, ultimately, on-device, paired with a hybrid approach: lightweight on-device models for latency-sensitive safety detection, with selective vision-language invocation for complex semantic understanding tasks when explicitly requested by users and subject to privacy filtering.

2.4.4 On-Device Optical Character Recognition Engines

Optical character recognition represents a critical enabling technology for visually impaired users, providing access to printed documents, product labels, signage, and digital text displays. For assistive applications, OCR engines must satisfy several technical requirements: (1) multilingual support including Arabic script recognition, (2) on-device inference capability to ensure privacy and offline functionality, (3) robustness to varying lighting conditions, text orientations, and image quality, and (4) computational efficiency compatible with mobile deployment. Google ML Kit [17], introduced in 2018, provides an on-device text recognition API leveraging deep learning models trained on diverse text datasets. The framework supports over 100 languages including Arabic, with specialized models for dense text (documents) and sparse text (scene text such as signs and labels). EasyOCR [18], an open-source alternative developed by JaidedAI, provides greater flexibility and customization capability, supporting 80+ languages with dedicated models for Arabic script and allowing fine-tuning on custom datasets. For Arabic OCR specifically, both engines face challenges stemming from Arabic script characteristics discussed earlier. Research by Ayadi et al. (2025) [2] demonstrates that specialized Arabic OCR models trained on regional datasets can achieve 89% accuracy compared to 68% for generic multilingual engines. Importantly, recent vision-language models subsume traditional OCR: rather than emitting raw character strings, a VLM reads text in context and reports only what is relevant in a natural spoken style. The implemented Wareeny reading pipeline takes this VLM-centric approach instead of a standalone OCR engine, which simplifies the stack and yields context-aware reading rather than verbatim transcription.

2.4.5 Hybrid Cloud-Local Processing Architectures

Contemporary assistive system design increasingly adopts hybrid architectures that strategically partition computational workloads between on-device (edge) processing and cloud-based inference. This architectural paradigm addresses the fundamental tension between capability, latency, privacy, and resource constraints that characterize mobile assistive applications. In hybrid architectures, latency-sensitive and privacy-critical tasks execute locally on the mobile device using lightweight, optimized models. These tasks typically include real-time obstacle detection, fall detection, traffic light recognition, and personal safety features requiring sub-300ms response times. Conversely, computationally intensive tasks benefiting from large-scale models—such as complex scene understanding, detailed document analysis, and interactive visual question answering—are selectively offloaded to a vision-language backend when network connectivity is available and user privacy preferences permit. The technical implementation of hybrid architectures requires careful orchestration: (1) intelligent task classification to determine local versus remote execution based on latency requirements, data sensitivity, and network availability; (2) privacy-preserving preprocessing that filters or downscales sensitive visual elements before transmission, as proposed by Martin & Moon (2025) [6]; (3) graceful degradation mechanisms that maintain core functionality when offline or in low-connectivity scenarios; and (4) battery-aware scheduling that balances processing demands against device energy constraints. For the proposed Wareeny system, the hybrid architecture prioritizes user privacy and offline capability while strategically leveraging a self-hosted vision-language model for enhanced semantic understanding when explicitly requested by users and subject to local filtering. This approach addresses the privacy-utility paradox identified earlier, providing comprehensive functionality without compromising data protection principles.

3

Chapter Three

System Analysis and Design

Chapter 3

System Analysis and Design

3.1 System Overview and Proposed Concept

The system overview described in this chapter outlines the architecture and functionality of the AI-powered solution developed to assist visually impaired individuals in navigating both indoor and outdoor environments. The proposed system leverages advanced artificial intelligence techniques, including computer vision, optical character recognition (OCR), object detection, and scene understanding, to analyze visual data captured through a camera-enabled device in real time. The core of the system is built upon a robust AI model trained to recognize obstacles, read text from the surrounding environment, and identify key objects and landmarks that are essential for safe and independent movement. Once the AI model is trained and optimized for performance, it is integrated into a user-friendly application designed specifically for visually impaired users. The application serves as the main interaction platform, providing audio and haptic feedback to convey meaningful information to the user. The AI model processes visual inputs, extracts relevant information such as text, objects, or navigation cues, and delivers clear and timely guidance to help the user understand their surroundings. This integration enhances the user's ability to move safely and confidently in complex environments, such as streets, buildings, and public spaces. By automating environment perception and information delivery, the system reduces dependence on external assistance and improves overall accessibility. The system model discussed in this chapter focuses on the interaction between the AI components and the application, the flow of data from image capture to user feedback, and how the generated insights are utilized to support real-time decision-making for visually impaired users.

3.2 Functional Requirements

The functional requirements of the Wareeny system are categorized into four main areas of assistance:

3.2.1 Indoor Assistance

FR-IN-1: **Indoor Object Detection:** The system shall detect indoor objects and estimate distance from them.

FR-IN-2: **In-Hand Detection:** The system shall detect specifically the object that is in the hand of the user, as to not get confused by other objects in the environment.

FR-IN-3: **Brand Detector:** The system shall identify product brands indoors.

FR-IN-4: **Currency Reader:** The system shall detect and read currency denominations.

FR-IN-5: **Privacy Mode:** The system shall detect sensitive data without speaking it aloud.

3.2.2 Outdoor Assistance

FR-OUT-1: **Outdoor Object Detection:** The system shall detect cars, bicycles, and other outdoor obstacles.

FR-OUT-2: **Outdoor Object Finder:** The system shall locate ATMs, benches, trash cans, and other helpful outdoor items.

FR-OUT-3: **Outdoor Reader:** The system shall quickly read shop names, billboards, and street signs.

FR-OUT-4: **Traffic Light Status:** The system shall detect traffic light colors for crossing guidance.

3.2.3 OCR Features

FR-OCR-1: **Read Document:** The system shall read documents, menus, books, and labels.

FR-OCR-2: **QA Mode:** Instead of just reading a document word for word, the system shall allow users to ask questions about it to get specific responses.

FR-OCR-3: **Danger Word Alerts:** The system shall notify users when dangerous keywords appear, such as “Wet Floor,” “Danger,” or “Caution.”

3.2.4 Audio and Haptics Service

FR-AH-1: **Danger Mode:** The system shall enable danger mode using voice command, gesture, or phone shake.

FR-AH-2: **Emergency Auto Call:** The system shall automatically call the user’s emergency contact in danger mode.

FR-AH-3: **Emergency Auto Location:** The system shall send the current location of the user to their emergency contact in danger mode.

FR-AH-4: **Directional Audio:** The system shall provide left or right ear cues to indicate object direction.

3.3 Non-Functional Requirements

The non-functional requirements define the quality attributes of the Wareeny system:

- **Performance:** The system shall provide real-time processing with minimal delay.
- **Accessibility:** The system shall offer full control using voice or simple gestures.
- **Privacy:** Local processing is preferred to ensure data protection.
- **Reliability:** The system must function in various indoor and outdoor lighting conditions.
- **Usability:** The system shall provide clear and consistent audio feedback with spatial cues.
- **Scalability:** The system shall support future expansion and improved AI models.

- **Security:** The system shall protect user data and stored personal images.
- **Power Efficiency:** The system shall reduce battery consumption during continuous operation.

3.4 System Environment

The proposed system is developed using a combination of modern hardware capabilities and advanced software technologies to ensure efficient, accurate, and real-time assistance for visually impaired users.

3.4.1 Hardware Environment

Table 3.1 summarizes the hardware requirements for the Wareeny system.

Table 3.1: Hardware Environment Specifications

Component	**Specification**
User Mobile Device	A modern smartphone capable of running AI-powered applications
Processor (CPU)	Multi-core processor (octa-core 2.0 GHz or higher recommended) to support real-time image processing and AI inference
Memory (RAM)	Minimum 2 GB RAM required; 4 GB or more recommended for smooth execution of deep learning models such as YOLO v8 and PyTorch-based frameworks
Camera	High-definition rear camera used for capturing images and video streams necessary for object detection, text recognition, and currency identification
Sensors	Built-in sensors such as accelerometer and gyroscope to enhance spatial awareness and provide navigation-related feedback
Connectivity	Active internet connection (Wi-Fi or 4G/5G) required for cloud-based services, including advanced scene understanding and text-to-speech processing

3.4.2 Software Environment

Table 3.2 presents the software technologies and frameworks utilized in the Wareeny system.

Table 3.2: Software Environment Technologies

Component/Tool	**Purpose and Description**
AI and Machine Learning Frameworks	
PyTorch & OpenCV	Used for model experimentation and computer-vision workflows such as detection, preprocessing, and localized AI prototyping
YOLO v8	Employed as the primary real-time object-detection architecture for navigation, stuff-finder, and Egyptian-pound banknote detection
Text Recognition (OCR)	
EasyOCR & Google ML Kit	Referenced for recognizing English and Arabic text in broader research; the implemented build instead reads text through a vision-language model
Gemma Vision-Language Model	Self-hosted multimodal model used for context-aware reading, scanning, scene description, and visual question answering
Voice and Audio Processing	
FlutterTTS	On-device text-to-speech for bilingual English/Arabic assistive narration
Whisper AI	Referenced for potential speech-to-text support in future voice-command and spoken-query interactions
Application Development Environment	
Flutter (Dart)	Used to develop a cross-platform mobile application compatible with both Android and iOS devices, ensuring accessibility and consistent user experience

3.5 System Architecture

The system architecture illustrates the high-level design of the Wareeny application, showing how different components interact to provide assistive functionality to the user. Figure 3.1 presents the overall architecture.

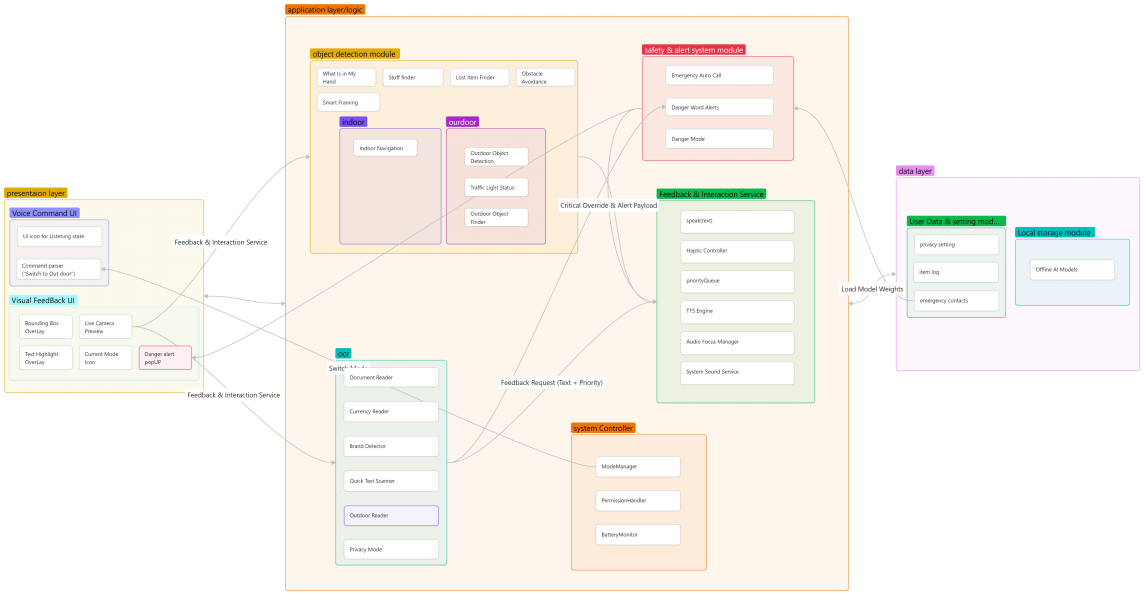


Figure 3.1: System Architecture Diagram.

3.6 Class Diagram

The class diagram illustrates the structural design of the Wareeny system, depicting the key classes, their attributes, methods, and the relationships between them. The diagram captures the core components of the application, including the user interface controllers, AI processing modules, service classes for object detection, OCR, and audio feedback, as well as data models for storing user preferences and detected objects. The relationships shown—such as associations, compositions, and inheritances—demonstrate how the system components collaborate to deliver seamless assistive functionality. This object-oriented design ensures modularity, maintainability, and scalability, allowing for future enhancements and the integration of additional AI models. The class structure is shown in Figure 3.2.

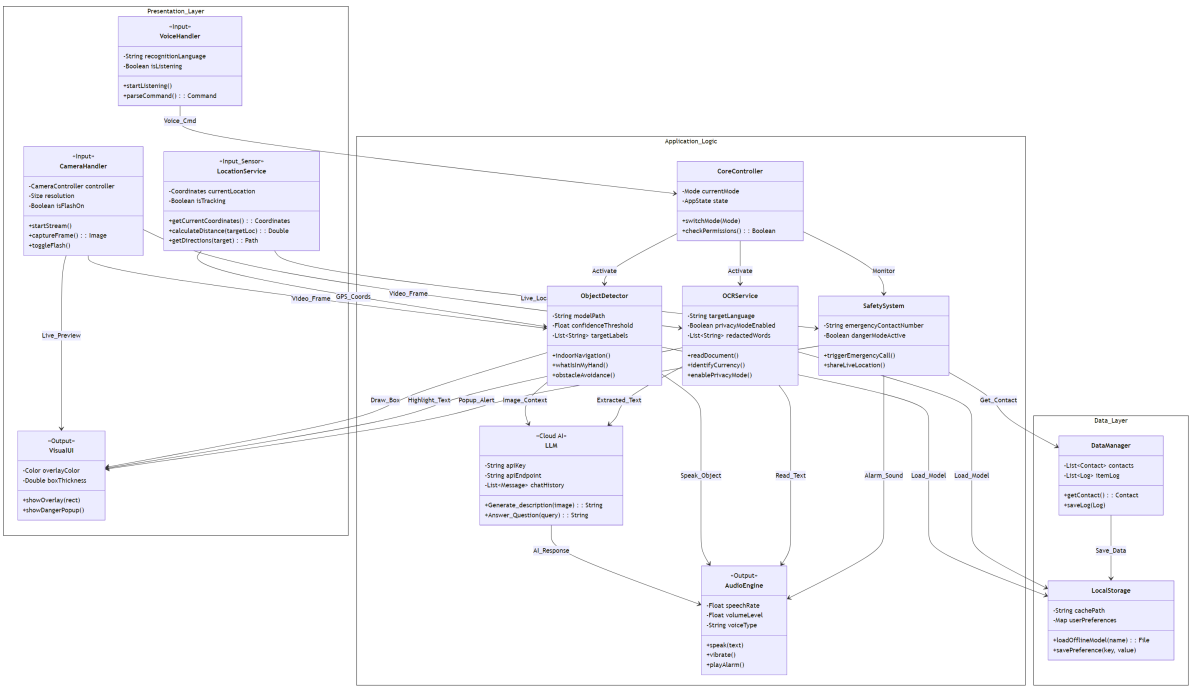


Figure 3.2: Class Diagram.

3.7 Use Case Diagram

The use case diagram illustrates the interactions between the user and the Wareeny system, showing the various functionalities available to visually impaired users. Figure 3.3 presents the system use cases.

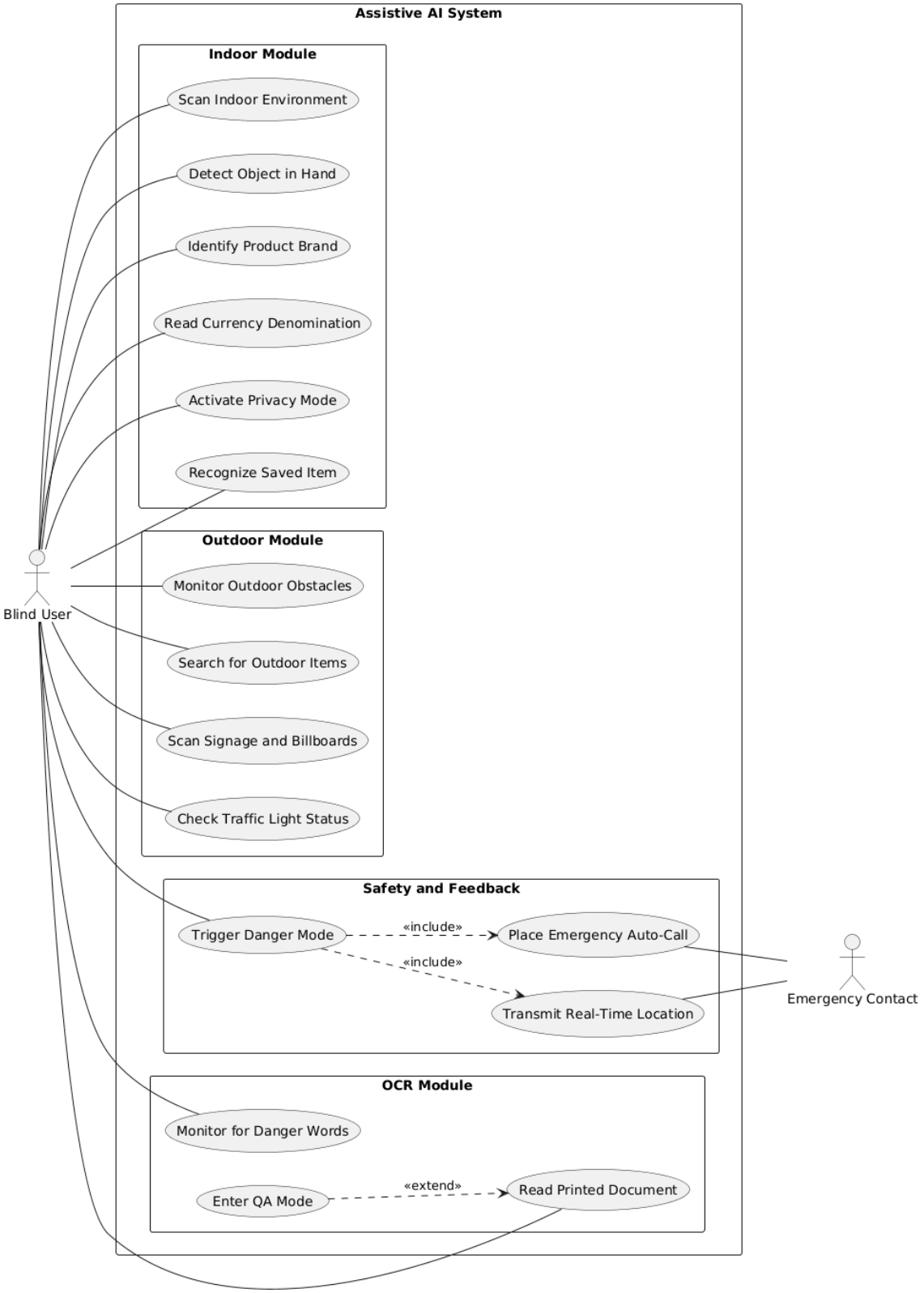


Figure 3.3: Use Case Diagram.

3.8 Sequence Diagrams

The following sequence diagrams illustrate the flow of interactions for the key features of the Wareeny system.

3.8.1 General Flow Diagrams

The three general flows summarize the dominant interaction patterns of the system: optical character recognition, object detection, and navigation.

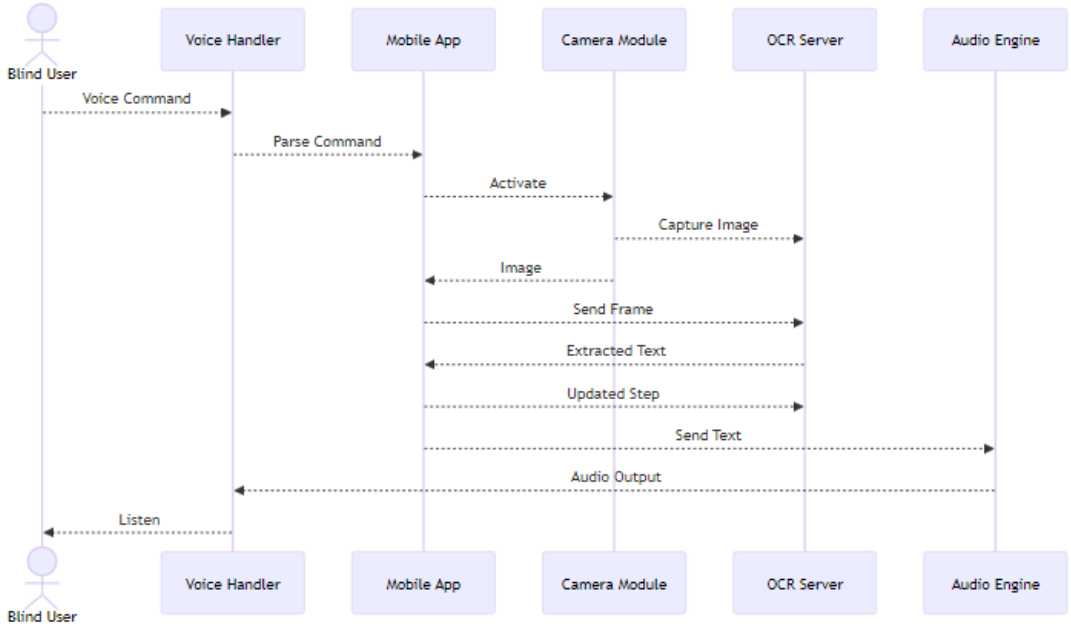


Figure 3.4: General Flow 1: OCR Processing.

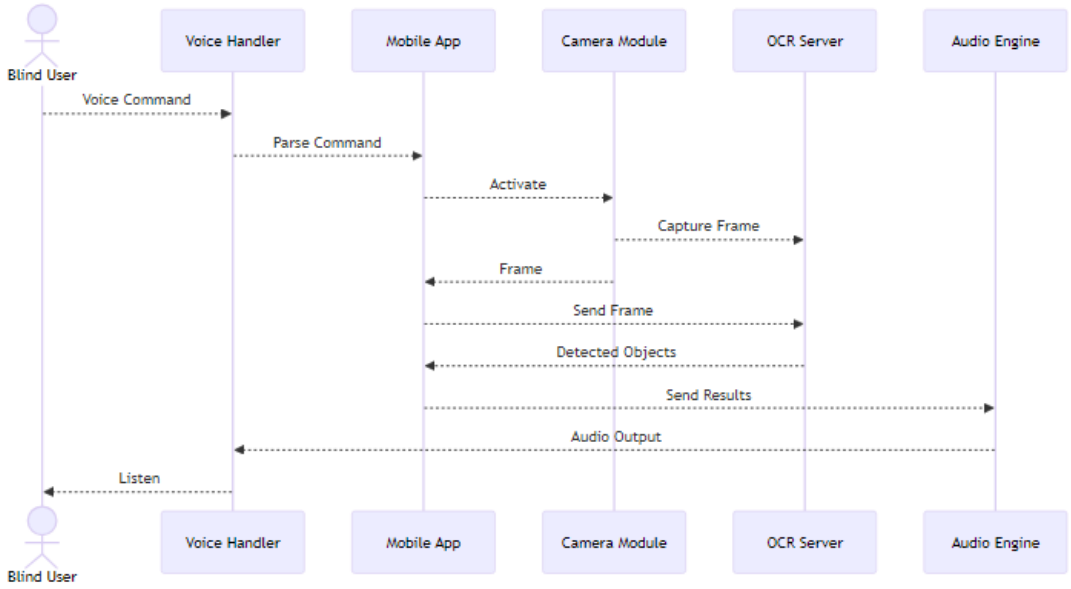


Figure 3.5: General Flow 2: Object Detection Processing.

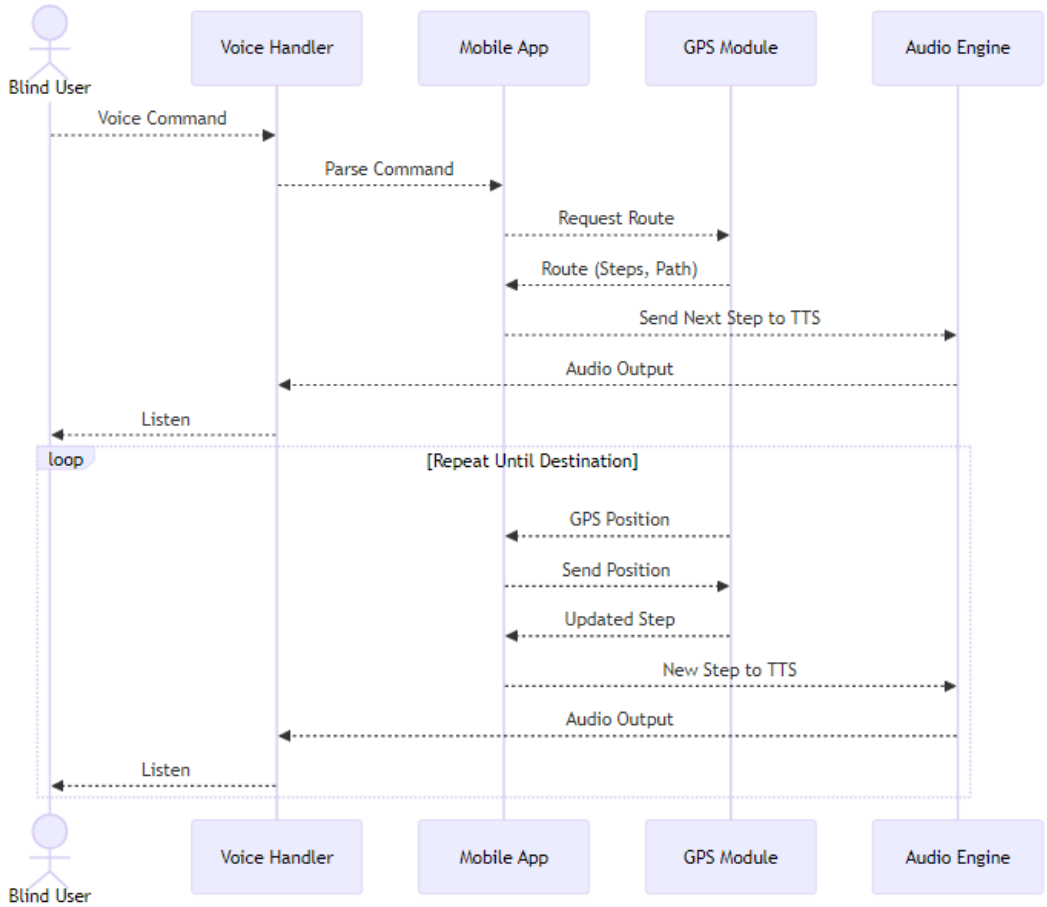


Figure 3.6: General Flow 3: Navigation Processing.

3.8.2 Indoor Features

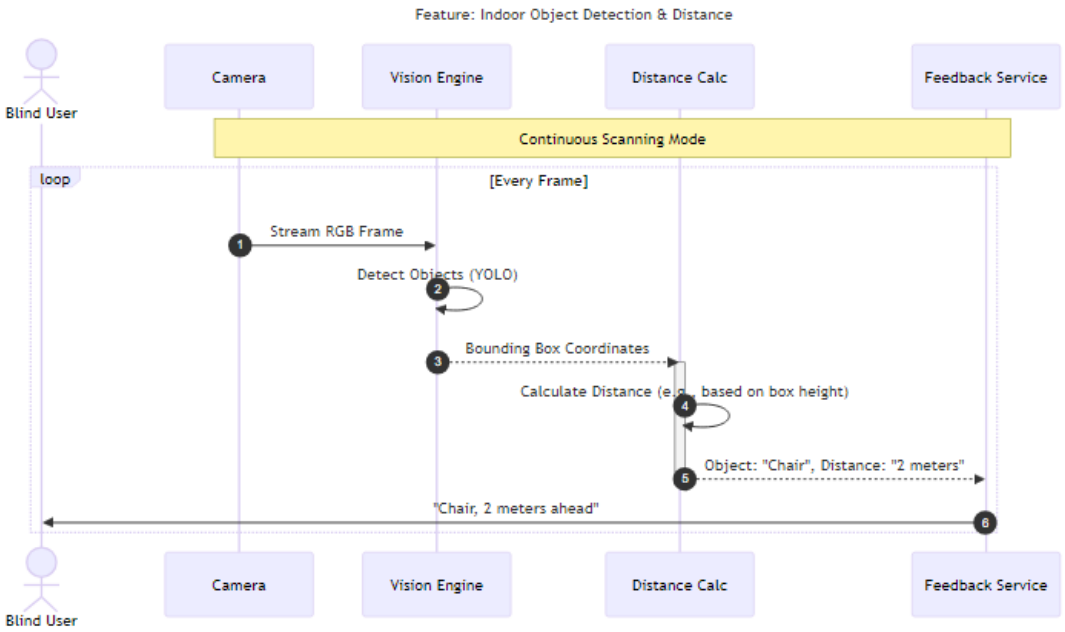


Figure 3.7: Sequence Diagram: Indoor Object Detection and Distance Estimation.

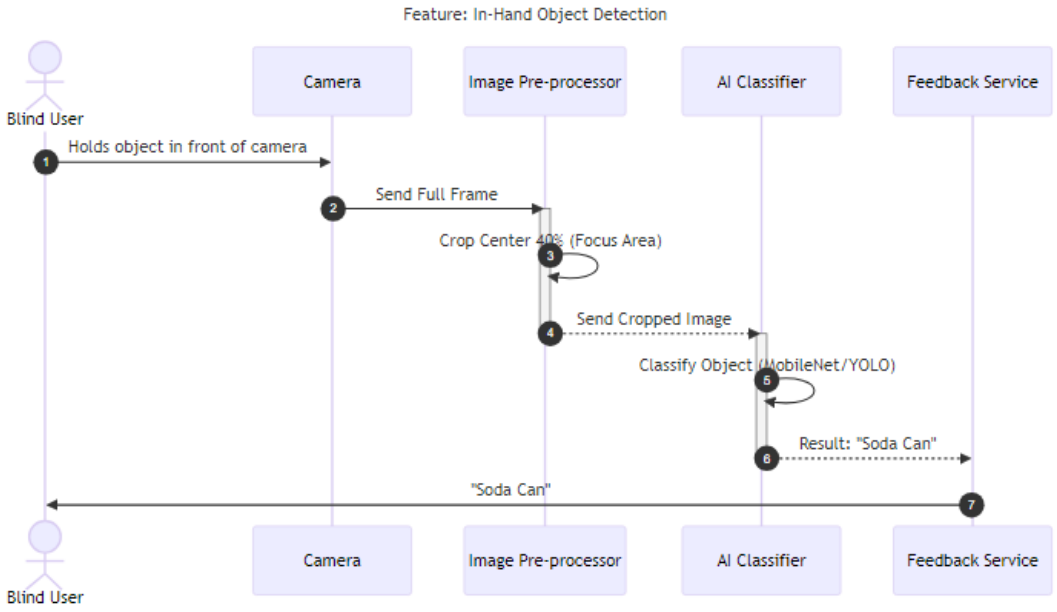


Figure 3.8: Sequence Diagram: In-Hand Object Detection.

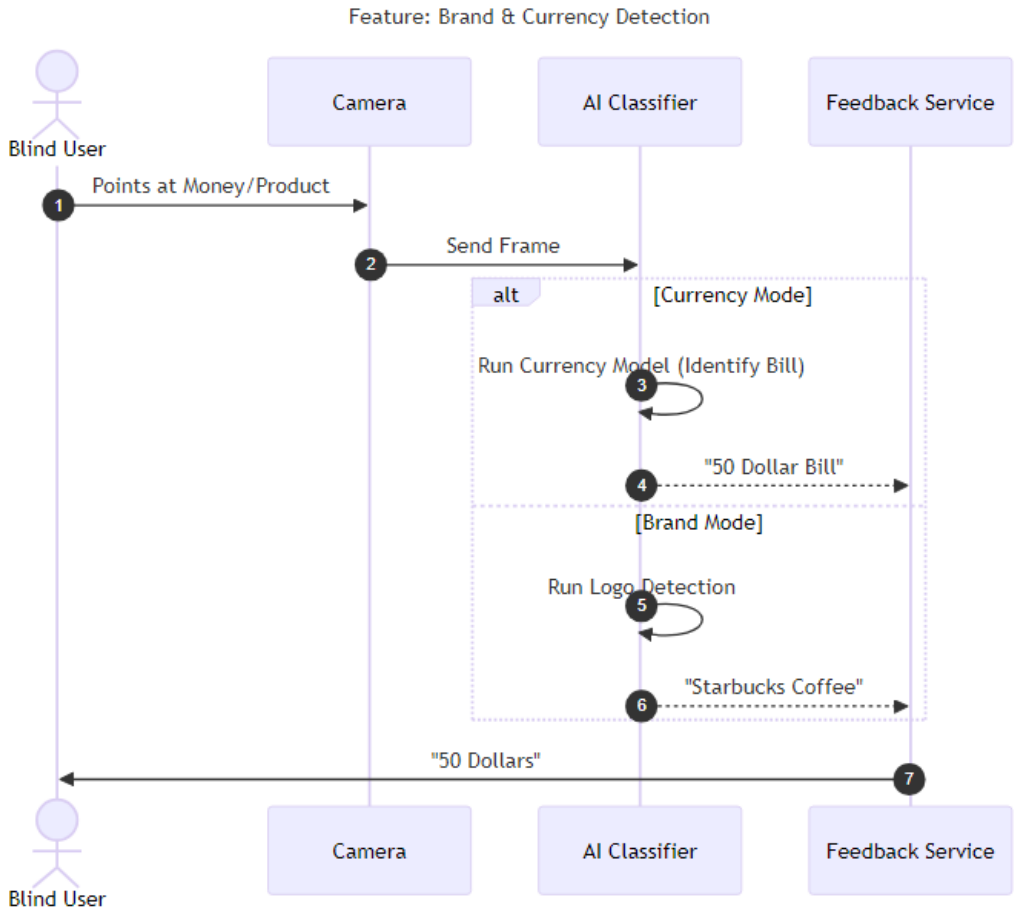


Figure 3.9: Sequence Diagram: Brand and Currency Detection.

3.8.3 Outdoor Features

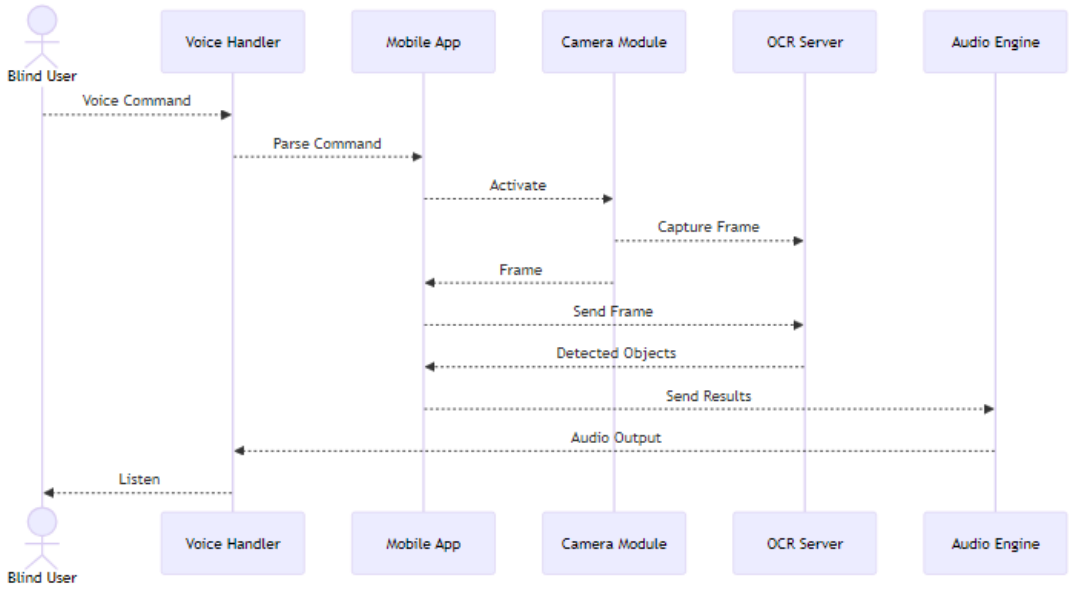


Figure 3.10: Sequence Diagram: Outdoor Object Detection.

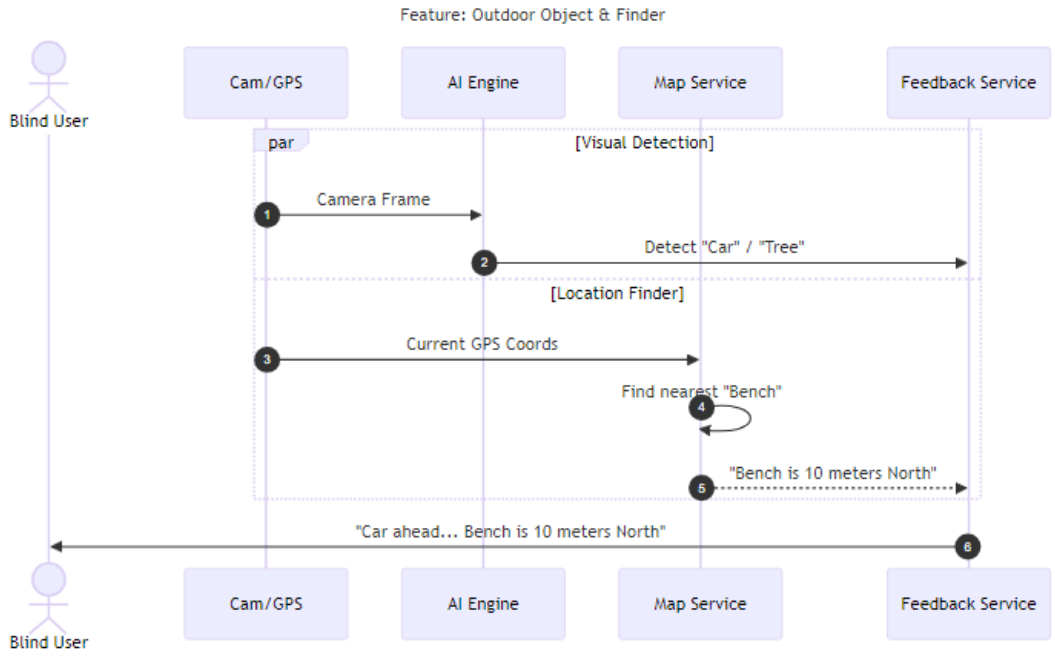


Figure 3.11: Sequence Diagram: Outdoor Object Finder.

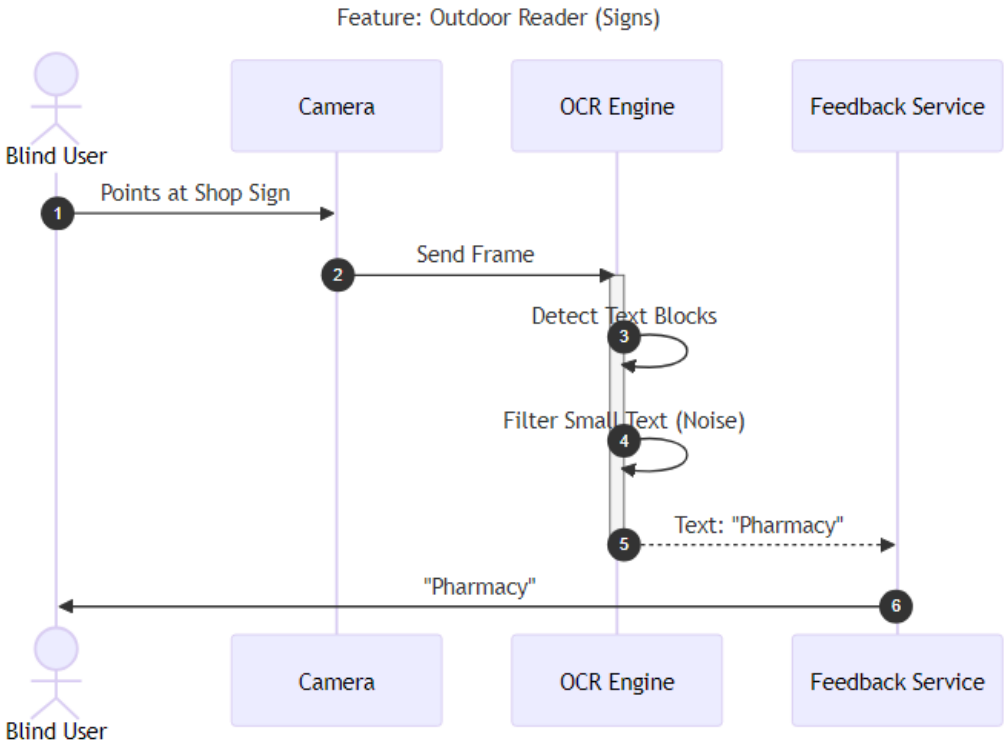


Figure 3.12: Sequence Diagram: Outdoor Reader for Signs.

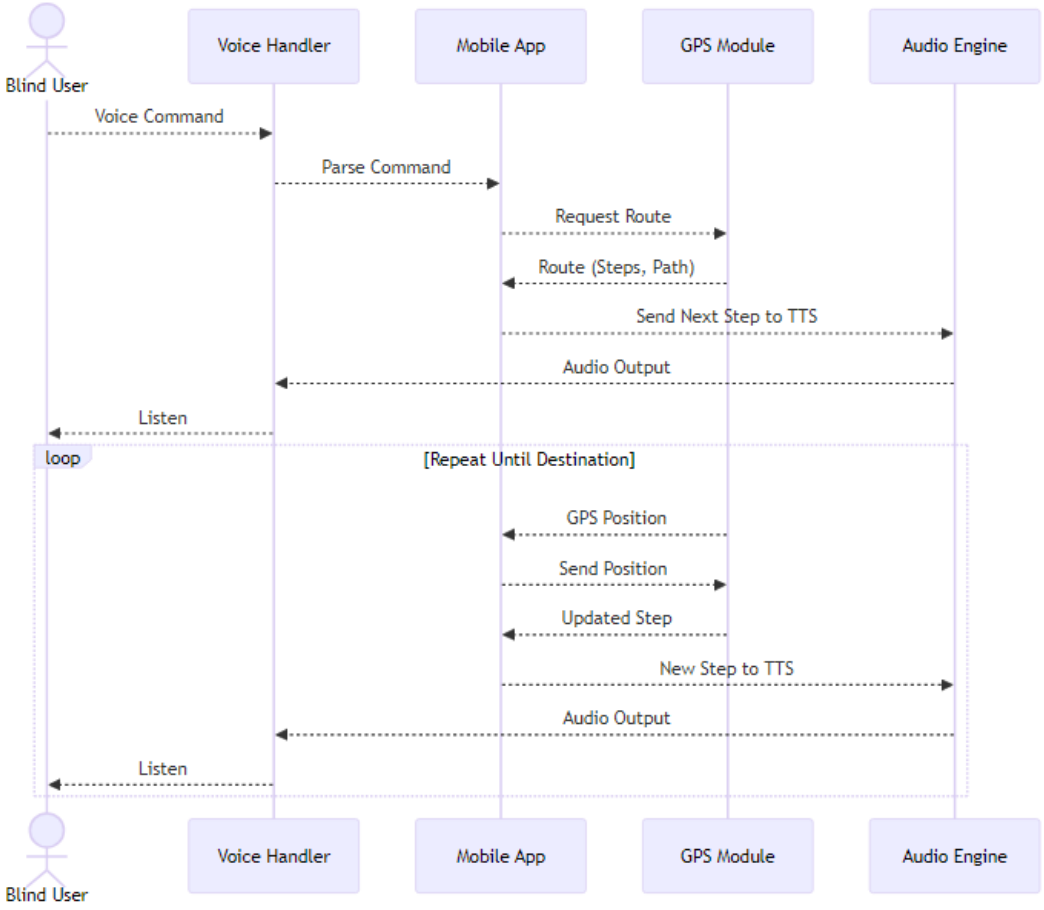


Figure 3.13: Sequence Diagram: Outdoor Navigation.

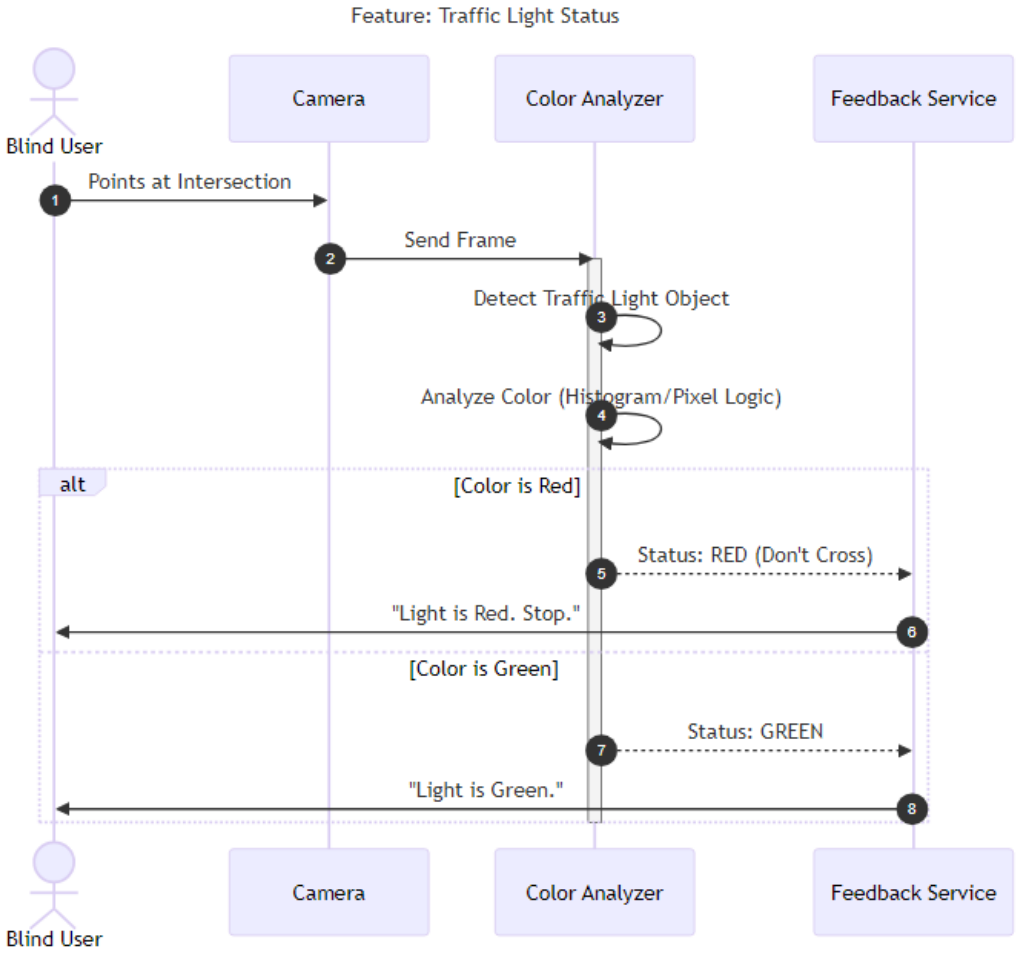


Figure 3.14: Sequence Diagram: Traffic Light Status Detection.

3.8.4 OCR Features

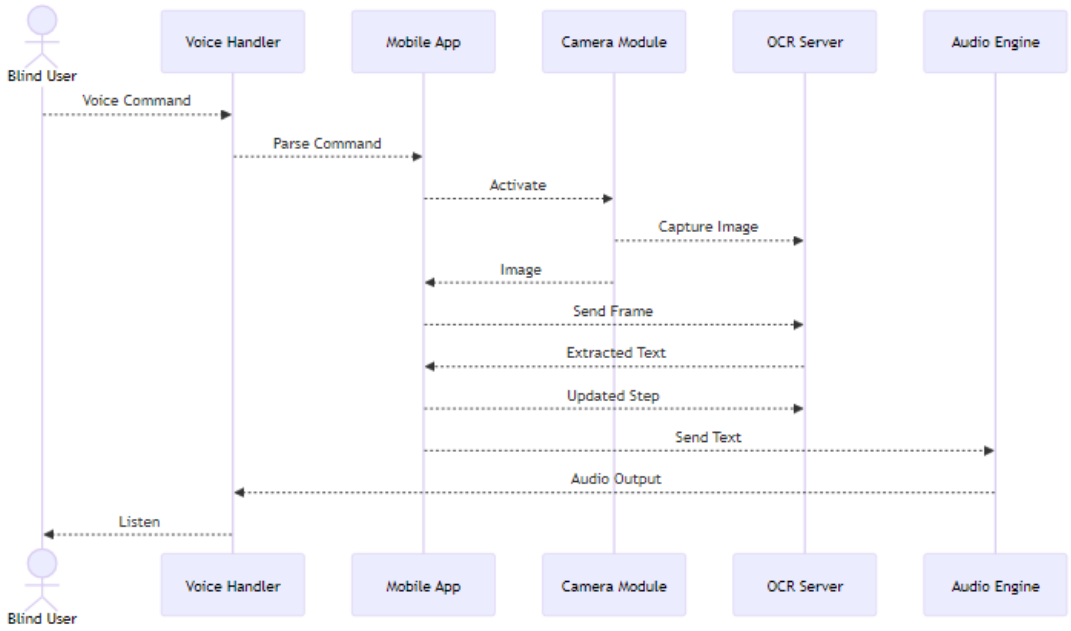


Figure 3.15: Sequence Diagram: Read Document.

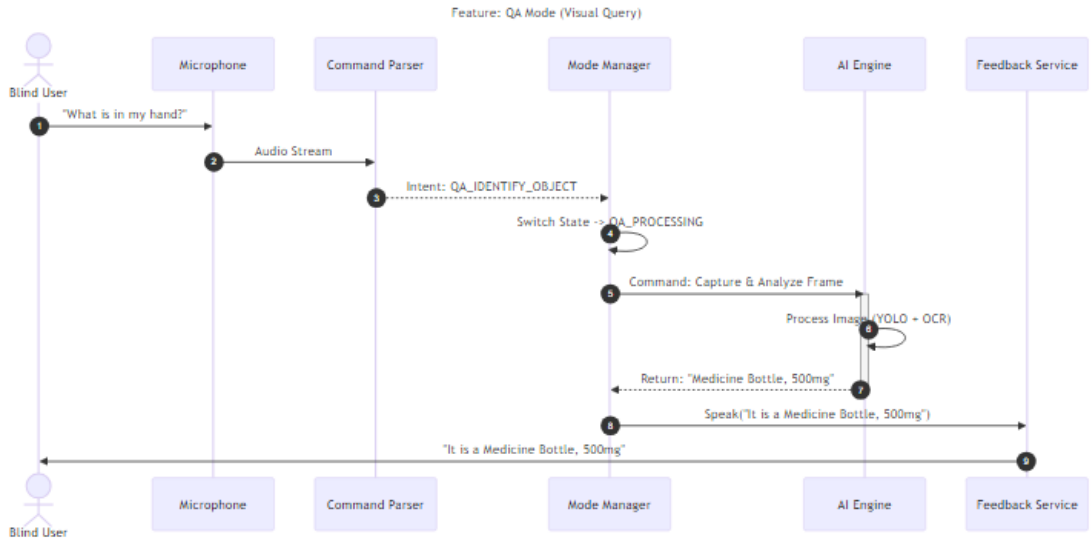


Figure 3.16: Sequence Diagram: QA Mode (Visual Query).

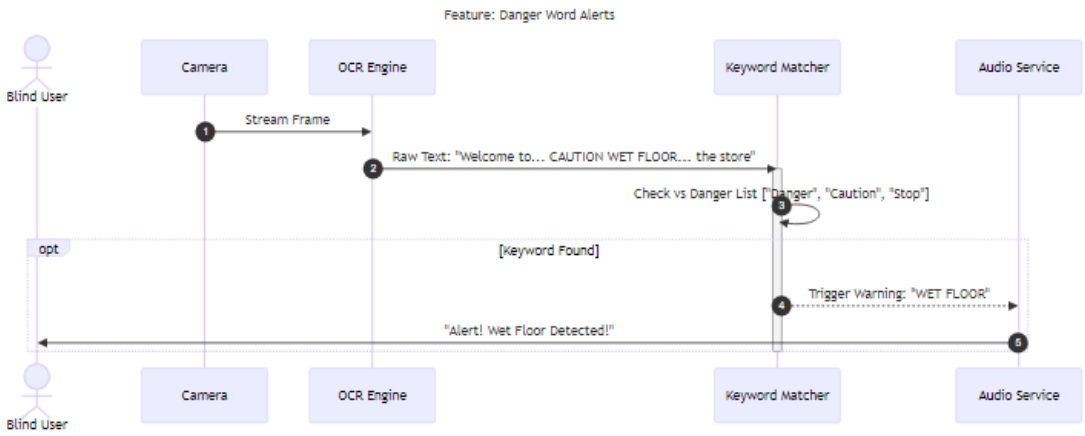


Figure 3.17: Sequence Diagram: Danger Word Alerts.

3.8.5 Safety and Emergency Features

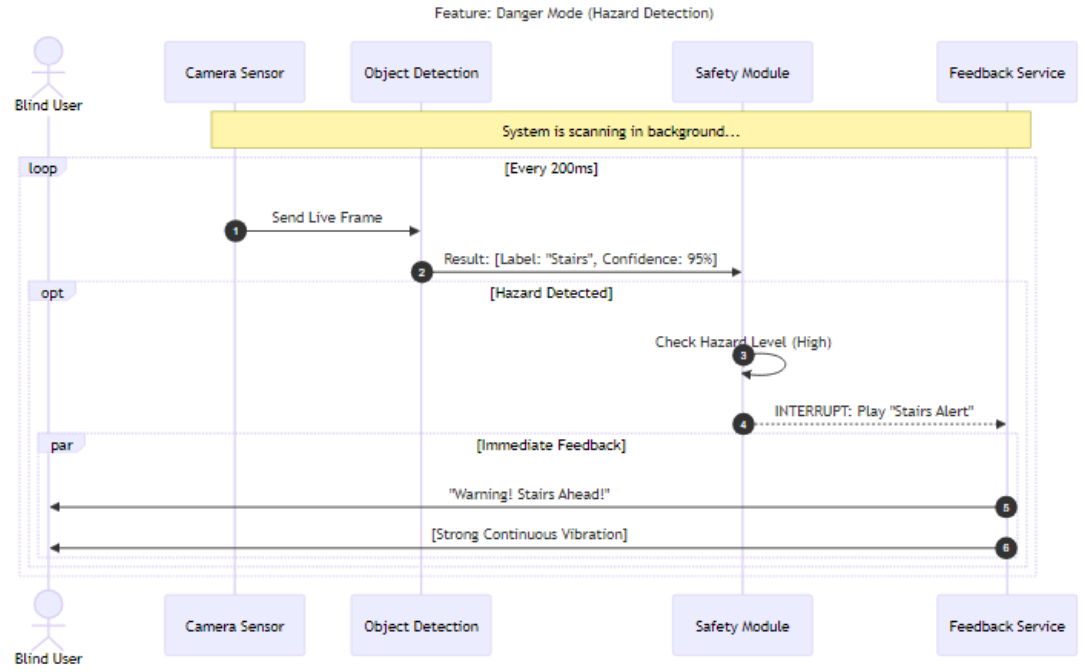


Figure 3.18: Sequence Diagram: Danger Mode (Hazard Detection).

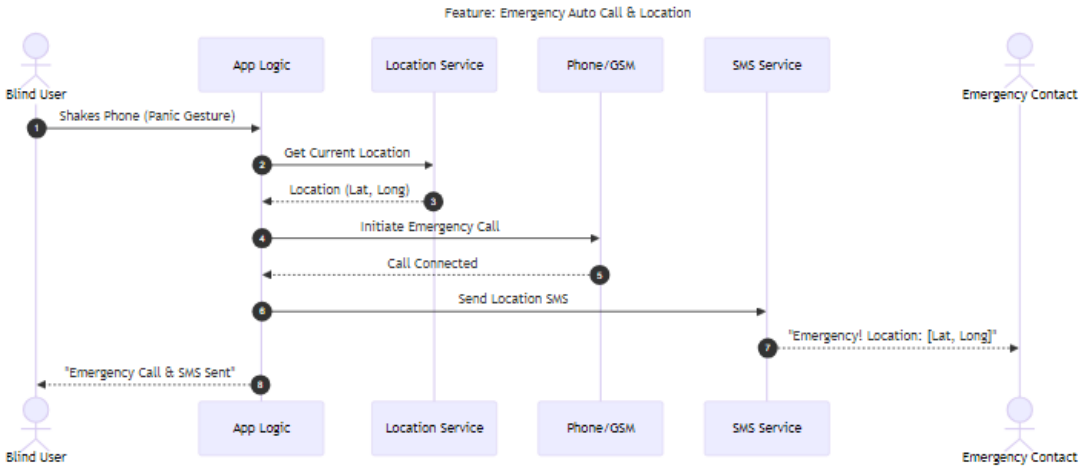


Figure 3.19: Sequence Diagram: Emergency Auto Call and Location Sharing.

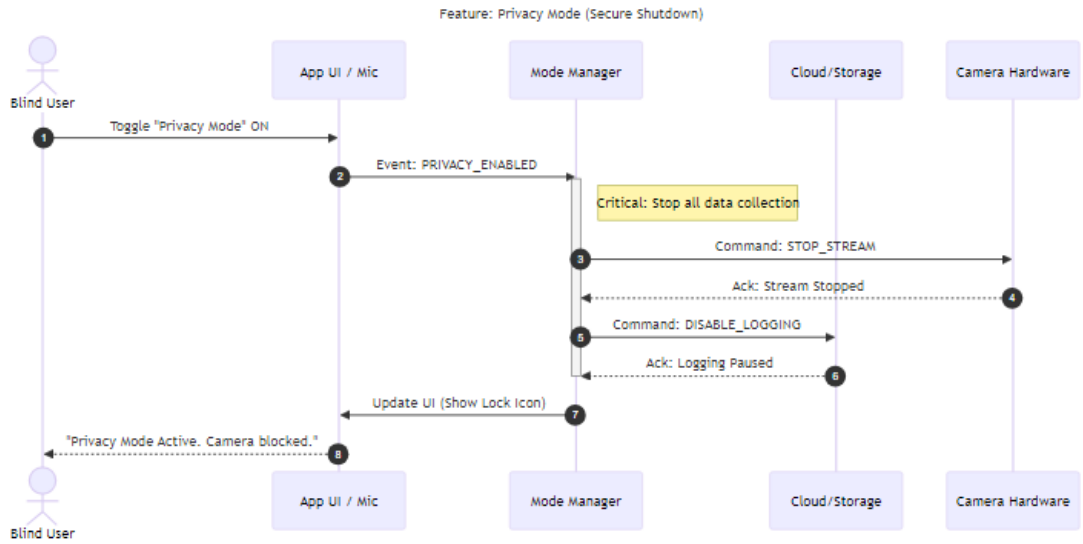


Figure 3.20: Sequence Diagram: Privacy Mode (Secure Shutdown).

3.8.6 Audio and Haptics Features

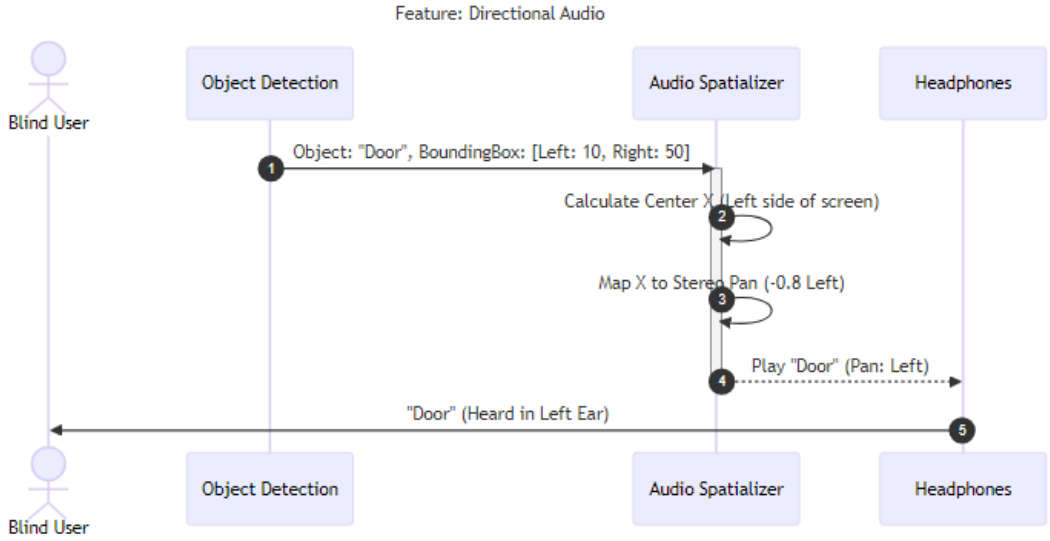


Figure 3.21: Sequence Diagram: Directional Audio.

3.9 Entity-Relationship Diagram

The Entity-Relationship Diagram (ERD) illustrates the database schema and data model of the Wareeny system. The diagram depicts the core entities required to store user information, system preferences, detected objects, emergency contacts, and usage history. The relationships between these entities—such as one-to-many and many-to-many associations—demonstrate how data flows and persists throughout the application lifecycle. This relational design ensures efficient data storage, quick retrieval of user-specific information, and maintains the integrity of personal data such as emergency contact details. The ERD serves as the foundation for implementing a robust and scalable database structure that supports all functional requirements of the assistive system. The data model is shown in Figure 3.22.

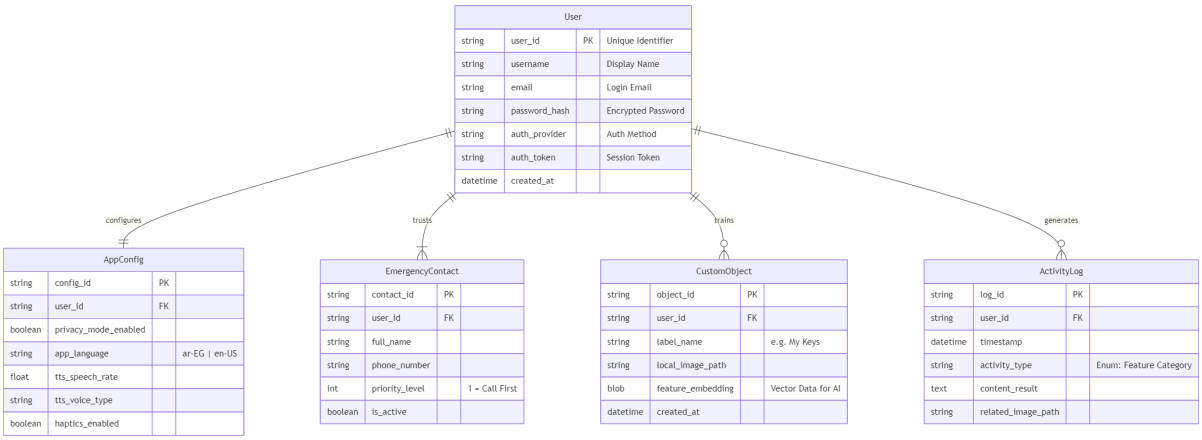


Figure 3.22: Entity-Relationship Diagram.

4

Chapter Four

System Design

Chapter 4

System Design

4.1 Introduction

This chapter presents the design of the Wareeny application as implemented in the project repository. Wareeny is an accessibility-oriented mobile assistant built with Flutter and centred on camera-driven interaction. In its present form the system combines a mode-based interface, a hybrid vision-language reading pipeline, real-time object-detection flows, an on-device Egyptian-pound banknote reader, a background emergency safety subsystem, spoken feedback, haptic alerts, bilingual English–Arabic localization, and persisted user settings. The implemented design is therefore not a conventional form-based mobile application but a sensor-driven assistive system in which perception services and accessibility feedback define the core user experience. Throughout this chapter, design decisions are described as they exist in the codebase, and the boundary between what is fully delivered and what is prepared for future work is stated explicitly so that the account remains accurate and verifiable.

4.1.1 Design Orientation

The design follows four guiding principles. First, interaction must remain usable without sight: every mode is reachable through two-finger gestures, exposes semantic announcements and custom screen-reader actions, and presents large touch targets. Second, high-cost perception logic is isolated behind controllers and service abstractions so that interface widgets stay lightweight and reusable. Third, non-real-time understanding tasks—scanning, reading, scene description, and image questions—are organized around a single shared vision-language architecture rather than isolated feature silos, so that a new understanding feature reuses existing services instead of duplicating inference logic. Fourth, real-time navigation and object-finding behaviour is delegated to local, on-device detection flows in order to keep latency low during active camera use and to avoid streaming a live video feed to any remote service.

4.1.2 Implemented Scope

The implemented application exposes six top-level modes: Home, Navigation, Stuff Finder, Scan, Read, and Settings. Navigation, Stuff Finder, Scan, Read, and Settings are delivered as complete, self-contained interaction paths with dedicated feature controllers. Home is a camera-centred entry surface rather than a full feature screen. Internally, the vision-language task model defines four tasks—scan, read, scene description, and ask-image—of which scan and read are fully wired to finished screens, while scene description and ask-image exist as service-level tasks ready to be surfaced. The Scan mode consolidates what were previously separate in-hand, currency, and brand con-

cepts into a single one-tap "identify whatever the camera is pointed at" interaction, with Egyptian-pound money reading handled ahead of the language model by a dedicated on-device banknote detector.

In addition to the visible carousel modes, the system runs an active background Emergency and Safety subsystem. This module remains armed during the application runtime, listening to accelerometer sensor data to detect user-shake events, enabling danger triggers without requiring visual search or manual screen clicks.

4.2 Development Tools and Technologies

4.2.1 Flutter and Dart

Flutter and Dart form the foundation of the application. Flutter provides the cross-platform widget system, theming model, animation support, semantics integration, and localization hooks used across the project, while Dart is used for screen composition, stateful feature controllers, dependency registration, bridge abstractions, and AI-service orchestration. This combination suits Wareeny because it supports rapid user-interface iteration while still allowing native platform integration through method and event channels where lower-level performance is required, such as haptic playback and the native object-detection runtime.

4.2.2 Visual Input and Camera Capture

The application uses two distinct visual-input paths, and the split is architecturally deliberate. For reading and scanning, the `camera` package opens the rear camera, shows a live preview, and captures a single still image on demand; the scan path requests a high-resolution preset specifically so that small printed banknote numerals remain legible. For navigation and stuff-finder, the `ultralytics_yolo` package provides a streaming camera view bound directly to YOLO inference. Reading and scanning need a deliberate still frame for interpretation, whereas navigation and object finding require continuous frame analysis; using the right capture model for each avoids paying streaming costs where a single frame suffices.

4.2.3 Vision-Language Reading Stack

The reading and scanning subsystem is built around the `VlmAssistant`, the single entry point that accepts an image path and a task definition and returns a stream of spoken-text events. It routes each request through a `ModelRouter`, a pure policy object that returns an ordered list of providers to try for a given task. Two providers implement the common `VlmProvider` contract: a `LocalVlmProvider` backed by an on-device vision-language runtime, and a `CloudVlmProvider` backed by a self-hosted Gemma backend. The assistant walks the providers in order, streams tokens from the first that can serve the request, and falls back to the next tier only when a provider declares itself unavailable before producing output. The task model defines a dedicated `currency` task; in the current build this task is served by the same vision-language pipeline as the other understanding tasks, while the `VlmProvider` contract is intentionally shaped so that a specialized on-device banknote provider can later be registered ahead of the language model without changing any screen or controller. Prompt construction is centralized in `VlmPrompts`, where each task carries a tuned, spoken-style system prompt, so that scan and read produce appropriately different behaviour while preserving a consistent assistive tone. A `SpeechQueue` receives streamed sentence-sized chunks and speaks them sequentially, allowing audio to begin before the full response has been generated.

4.2.4 Real-Time Object Detection Stack

Real-time detection is abstracted behind the `ObjectDetectionSource` interface. The concrete implementation in active use is `UltralyticsYoloDetectionSource`, which emits streaming detection frames and selects bundled

model assets according to the active feature mode—a general YOLOv8n detector for navigation and a custom-trained YOLOv8n detector for stuff finding. Detection frames are normalized through a `DetectionCoordinateMapper`, which corrects for sensor rotation and mirroring, before higher-level logic such as the `NavigationRiskEngine` or the `StuffFinderController` interprets them. The same single-stage detection approach is the basis on which a dedicated single-shot Egyptian-pound banknote detector is planned: it would run a custom-trained YOLOv8n banknote model on individual captured stills rather than on a live stream, since money reading is tap-to-capture rather than continuous. This detector is not yet bundled in the current build, and currency reading is presently handled by the vision-language pipeline described above.

4.2.5 Localization, Speech, and Preference Persistence

Flutter localization delegates and a custom `AppLocalizations` class provide bilingual English and Arabic user-facing strings. Audio output is handled by the `AudioHapticsBridge`, which wraps `flutter_tts`, prefers an Egyptian-Arabic (`ar-EG`) voice when one is installed, and configures speech rate, pitch, and completion callbacks; the same bridge also exposes categorized haptic signals. Persisted user settings are managed by the `SettingsController`, which stores the locale, speech rate, and haptics preference in `SharedPreferences`. This gives the application a lightweight persistence model suitable for personalization without introducing premature relational-database complexity.

4.3 Architectural Evolution: Detection-and-OCR versus a Vision-Language Pipeline

The current architecture is the result of a deliberate design pivot made during development. The initial plan followed the conventional assistive-application pattern: a YOLO-family object detector would handle every recognition task, and Google ML Kit would handle every text-reading task. This combination is fast and fully on-device, but as the requirements matured it proved structurally mismatched with the accessibility goals of the project, and the understanding layer was re-centred on a vision-language model (the self-hosted Gemma family) instead.

4.3.1 Motivation for the Pivot

Three limitations drove the change. First, a supervised detector such as YOLO can only recognize the finite set of classes it was trained on; any object outside that set is simply invisible to it, which is unacceptable for an assistant whose users point the camera at arbitrary, unpredictable objects. A vision-language model, by contrast, performs strong zero-shot recognition and can describe objects it was never explicitly trained to detect. Second, a classical OCR engine such as ML Kit emits a flat, verbatim character stream with no understanding of layout, salience, or intent; it cannot distinguish a heading from a footer, cannot answer a question about a document, and tends to overwhelm a blind user with undifferentiated text. Mekonnen et al. (2025) demonstrate this ceiling concretely: a state-of-the-art offline assistive system using Tesseract OCR achieves 89.7% word accuracy on clean printed English text under optimal lighting, but this falls to 73.9% at normal walking speed, and the engine provides no support for Arabic script or semantic understanding of what it reads [20]. A vision-language model reads text *in context*, reporting only the facts that matter in a natural spoken style, and handles Arabic and mixed-language documents without a separate language-specific engine. Third, a detector produces bounding boxes, not answers: it cannot support Visual Question Answering (VQA), which the literature (Huh et al., 2024 [1]) identifies as the interaction blind users most prefer. These three gaps—closed-set recognition, context-free OCR, and the absence of reasoning—together motivated consolidating the understanding tasks behind a single vision-language pipeline, while retaining YOLOv8n only for the latency-critical, closed-set tasks (navigation and stuff finding) where its speed is an asset and its fixed class set is acceptable.

4.3.2 Comparative Justification

Table 4.1 preserves the conceptual trade-off between the two design directions, while Table 4.2 adds representative published statistics for speed, accuracy, and user outcome. The values are included to justify the architectural decision and are drawn from the literature reviewed earlier rather than from direct benchmarking of the implemented repository.

Table 4.1: Classical Detection/OCR versus the Vision-Language Pipeline for Accessibility

Dimension	**YOLO / ML Kit (initial plan)**	**Vision-Language Model (adopted)**
Object recognition scope	Closed-set: only trained classes (e.g. 80 COCO classes [11]); effectively 0% recall on any untrained object	Open-vocabulary, zero-shot: recognizes and describes objects outside any fixed label set
Text reading	Verbatim OCR; no layout or salience awareness; reads page furniture indiscriminately	Context-aware reading; distinguishes body from headers/footers and reports only salient content
Reasoning / interaction	Bounding boxes and class labels only; no question answering	Supports Visual Question Answering (VQA) and multi-turn queries about the image
Output for blind users	Raw labels and full text dumps, prone to cognitive overload	Concise, natural-language spoken answers tuned to user intent
On-device latency	Very low; real-time streaming (~15 FPS, YOLOv8n [15])	Higher per-query latency; suited to deliberate, tap-to-capture tasks
Role in final design	Retained for latency-critical, closed-set tasks (navigation, stuff finding)	Adopted for all open-ended understanding (scan, read, scene, VQA)

Table 4.2: Numerical Performance Comparison: Classical Pipeline vs. Wareeny's Hybrid Approach

Metric	**Classical Pipeline**	**Wareeny Approach**	**Gain**	**Ref.**
Navigation speed (FPS)	12.8–15.2	12.8–15.2 (preserved)	—	[4], [5]
OCR accuracy – Arabic handwriting (%)	68	89 (VLM contextual)	+21 pp	[2]
OCR accuracy – dynamic scene (%)	73.9	No fixed ceiling	—	[20]
Currency recognition – controlled (%)	72	94 (localized model)	+22 pp	[3]
Currency recognition – worn notes (%)	72	87 (localized model)	+15 pp	[3]
User task completion vs. static captioning	baseline	+40%	+40%	[1]

Table 4.2 therefore compares the two directions more directly. The classical pipeline remains strong in one area: speed. That is why Wareeny keeps detector-based processing for navigation and preserves the real-time 12.8

15.2 FPS operating range reported in the literature [4], [5]. Where the adopted Wareeny design improves is in understanding quality and user usefulness. Instead of relying on an OCR-only path whose robustness can fall to 68% on challenging Arabic text [2], the system shifts reading into a contextual VLM pipeline, motivated by Arabic-reading results that rise from 68% to 89%, or by 21 percentage points, when moving beyond the generic OCR baseline [2]. Instead of depending on generalized visual models for region-specific tasks, the architecture leaves room for localized Egyptian-currency recognition, where published results rise from 72% to 94% in controlled conditions [3]. Most importantly, the interaction model aligns with evidence that interactive VQA-style assistance can improve task completion by 40% over static captioning [1]. This is the numerical reason the project adopts a hybrid architecture rather than a purely classical one.

4.4 System Architecture

The application is organized into five cooperating layers: presentation, application control, AI services, backend integration, and persistence. Dependencies flow downward—screens depend on controllers, controllers depend on shared services, and services depend on bridges and providers—while no lower layer reaches back into a higher one. Cross-cutting dependencies are wired once during startup by a `ServiceRegistry` and exposed to the widget tree through an inherited `AppScope`, which keeps construction logic out of individual screens.

4.4.1 Presentation Layer

The presentation layer consists of the `NavigationShell` and the feature screens it hosts: `ReadScreen`, `ScanScreen`, `NavigationScreen`, `StuffFinderScreen`, and `SettingsScreen`. Reusable widgets—`CameraTopBar`, `ModeCarouselBar`, `TwoFingerSwipeDetector`, status pills, side-action controls, and mode-specific overlays—provide a consistent visual language across modes. The presentation layer is deliberately thin: it captures gestures, renders current state, and delegates all feature behaviour to dedicated controllers.

4.4.2 Application Control Layer

The application control layer contains the stateful controllers that coordinate each feature. The `VlmReadController` drives both Scan and Read: it manages the capture-to-speech state machine, token streaming, the "thinking" cue that covers first-token latency, and—for Read—the stop, repeat, skip, and cancel controls. The `NavigationController` consumes streaming detection frames, applies geometric risk rules with confirmation windows and cooldowns, and triggers directional speech and haptics. The `StuffFinderController` manages target selection, detector startup, match filtering, and cooldown-gated spoken announcements. The `SettingsController` manages persistent configuration and notifies the interface whenever the locale, speech rate, or haptics preference changes.

4.4.3 AI Service Layer

The AI service layer holds the reusable logic beneath the controllers. On the vision-language side it includes the `VlmAssistant`, the `ModelRouter`, the centralized `VlmPrompts`, and the three provider implementations. On the detection side it includes the `ObjectDetectionSource` abstraction, the `DetectionCoordinateMapper`, the detection data models, the `BanknoteDetector`, and the navigation geometry and risk scoring in `NavigationRiskEngine`. Keeping this logic in a shared layer is what allows new assistive flows to be added by reusing services rather than duplicating inference inside screens.

4.4.4 Backend Integration Layer

The backend integration layer is represented by the `CloudVlmProvider`. This provider probes a remote `/health` endpoint, downscales and re-encodes the captured image off the main isolate to bound upload size and first-token latency, sends a multimodal request to `/vlm/generate`, and parses a Server-Sent Events stream so that tokens can be spoken as they arrive. It also recognizes a streamed error payload from the backend and converts it into an "unavailable" signal so that the assistant can fall back or report cleanly rather than appearing to produce an empty reading. The backend service itself is external to this repository, but the client-side contract—endpoints, request shape, and streaming format—is fully encoded in the application.

4.4.5 Persistence Layer

The persistence layer is intentionally lightweight. `SharedPreferences` stores the long-lived personalization keys, while all transient perception state—streamed reading text, the active speech queue, the latest detection frame, the current navigation assessment, and the selected stuff-finder target—is held in controller memory and is meaningful only within the current session. Wareeny therefore uses a mixed persistence model by design: durable key-value settings for personalization and in-memory runtime objects for live camera sessions.

4.5 Application Map

4.5.1 Startup and Home Flow

Application startup begins in a `bootstrap` routine, which initializes the widget binding, loads environment configuration through `AppConfig`, builds the `ServiceRegistry` asynchronously (restoring settings from `SharedPreferences` along the way), and eagerly initializes text-to-speech so that the first spoken prompt is not delayed. The user is then placed into the `NavigationShell`. From the shell, Home acts as a camera-centred entry surface rather than a form-driven dashboard, and the user moves across modes through the carousel or two-finger horizontal swipes.

4.5.2 Navigation Mode Flow

In Navigation mode the shell opens `NavigationScreen`, instantiates a `NavigationController`, and connects it to a YOLO streaming view. Detection frames are oriented, passed into the `NavigationRiskEngine`, and converted into caution, danger, low-light, or path-clear feedback. The risk engine evaluates each object against a forward path region, weighing confidence, box area, and overlap with the path; the controller then stabilizes that signal through short confirmation windows (shorter for danger than for caution) and rate-limits output with separate speech and haptic cooldowns, so that warnings are timely without becoming repetitive. Spoken guidance is issued in Egyptian Arabic—for example, a left-offset obstacle prompts the user to step right and a close frontal obstacle prompts a stop—and a path-clear confirmation is spoken only after several consecutive clear frames. Indoor and outdoor submodes are present in the interface and currently share the same detection pipeline, which leaves room for later environment-specific tuning without restructuring the screen.

4.5.3 Stuff Finder Mode Flow

In Stuff Finder mode the user first selects a target from a picker that merges custom assistive classes—such as wallet, glasses, and keys—with standard COCO object classes. Once a target is chosen, the `StuffFinderController` starts the detector, selects the appropriate custom model path, receives streaming frames, filters detections down to the selected label, and updates a status indicator with the current match state. Spoken feedback announces the

4.5.4 Scan Mode Flow

Scan mode is the application's one-tap identification path, implemented by `ScanScreen` over the shared `VlmReadController`. A single tap captures a high-resolution still, which is submitted as a scan task. The `ModelRouter` prefers the on-device vision-language model for privacy and offline use and falls back to the self-hosted backend when the local model is unavailable and the network is reachable. The model identifies the subject as a packaged product read off its label, a sign or document reduced to the one or two facts that matter, a banknote whose denomination is reported, or a plain object. Because Scan targets blind users, it offers no playback controls; instead it exposes a "repeat result" gesture and screen-reader action so a missed answer can be heard again. The routing layer is structured so that a dedicated on-device banknote detector can later be inserted ahead of the language model for this path, improving denomination accuracy and enabling fully offline money reading; that detector is reserved for future work.

4.5.5 Read Mode Flow

In Read mode the user captures a still image of a document, which is submitted as a read task. Routing prefers the on-device vision-language model for privacy and offline use and falls back to the self-hosted backend when the local model is unavailable and the network is reachable. The read prompt instructs the model to read the main body faithfully and in order while skipping page furniture such as headers, footers, and page numbers, so the result is a faithful reading rather than a summary. Streamed text is buffered into speakable sentences and spoken progressively, and the screen keeps stop, repeat, skip, and cancel controls available until the speech queue has fully drained, because generation typically outruns playback.

4.5.6 Settings Flow

Settings mode operates without the live camera and provides language, speech-rate, and haptics controls presented as glass-style configuration cards in the same visual language as the rest of the application. Every change is written to `SharedPreferences` immediately and propagated through the widget tree, so subsequent launches reproduce the user's preferred accessibility profile.

4.5.7 Emergency and Safety Flow

The safety flow is managed as an active background service by the `EmergencyController`. Armed on application bootstrap, it listens to acceleration events via the `ShakeDetectorService`. When a qualifying device shake is detected, the controller halts active presentation, triggers a semantic announcement, and pushes the modal `EmergencyOverlay` onto the screen. This starts a 5-second countdown timer. If the user does not tap the cancel target, the controller executes a multi-stage emergency dispatch: it (1) fetches device GPS coordinates via the `LocationService`, (2) automatically dials the primary emergency contact via `flutter_phone_direct_caller`, (3) silently formats and transmits an SMS message containing the location link natively (Android) or opens a prepared draft (iOS), and (4) invokes a WhatsApp intent with a pre-filled safety message for redundant contact warning.

4.6 UI/UX Design

4.6.1 Accessibility Interaction Model

The interaction model is designed around non-visual control. Two-finger horizontal swipes switch top-level modes, and feature screens attach custom semantics actions so that screen-reader users can jump directly to any other mode or invoke feature actions without relying on visual controls. Live status values are exposed as semantic live regions,

and major mode changes are announced aloud. Accessibility is therefore not an added layer over a visual interface; it is the primary navigation model of the system.

4.6.2 Visual Design Language

The visual layer uses a dark full-screen camera background with glassmorphism overlays, electric-green highlights, rounded cards, and centrally placed status elements, applied consistently across the top bar, mode carousel, settings cards, reading controls, and status pills. Even though the target user may not depend on vision, a clean and stable visual layout remains valuable for caregivers, demonstrations, testing, and hybrid low-vision use.

4.6.3 Screen Designs

Figures 4.1–4.3 present the principal screen designs of the implemented application.

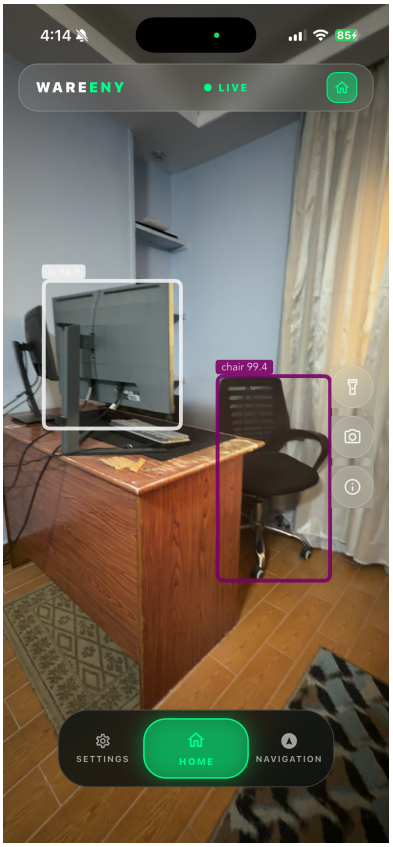


Figure 4.1: Home and Scan entry surface with the camera preview, mode carousel, and top status bar.

4.6.4 Bilingual Experience

Wareeny supports English and Arabic at the application level. Locale changes propagate through `MaterialApp` and `AppLocalizations`, so labels, hints, and spoken announcements adapt to the selected language, and Flutter's bidirectional layout handling provides right-to-left presentation when Arabic is selected. Spoken output is matched to content: the navigation safety messages are issued in Egyptian Arabic, the money reader localizes the words around the amount while keeping the digits themselves, and the reading model answers in the language of the text it is reading.

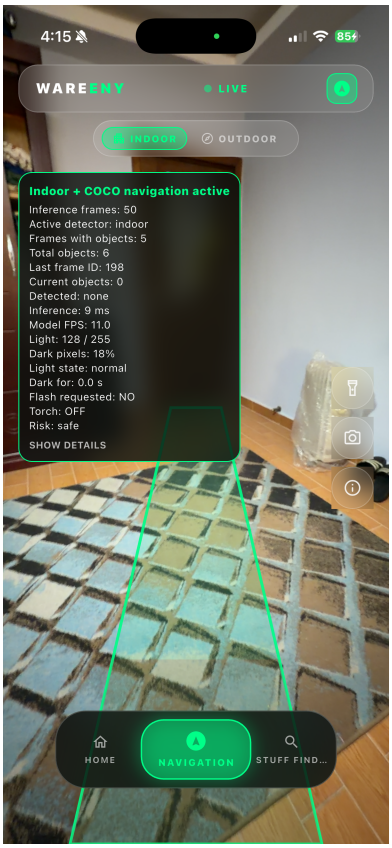


Figure 4.2: Navigation mode showing live detection overlays and directional status indicators.

4.6.5 Feedback Strategy

Feedback is intentionally multimodal. Visual overlays indicate live status, targets, and reading results; text-to-speech communicates prompts and findings; and categorized haptic signals support confirmation, navigation cues, obstacle warnings, and danger states. The audio layer incorporates completion and interruption handling so that feature controllers can manage reading state predictably—knowing, for example, when the queue has finished speaking. This combination makes the design more robust than dependence on any single output channel.

4.6.6 Safety UI Overlay and Onboarding

The interface integrates a dedicated safety layer consisting of the `EmergencyOnboardingScreen` and the modal `EmergencyOverlay`. The onboarding screen is shown on very first launch to explain the shake-to-activate mechanism using clear high-contrast buttons and spoken alerts, guiding the user to enter emergency contacts. The `EmergencyOverlay` is rendered with a dark translucent crimson gradient background showing a large pulsing red circular ”SOS” button at the center. This design relies on standard animation parameters to communicate urgency visually while maintaining a full-screen gesture detector, allowing the user to abort the dispatch by tapping anywhere on the screen.

4.7 Data Persistence

4.7.1 Persistence Philosophy

Wareeny is a locally-operated assistive application. It does not maintain a user account, does not transmit personal

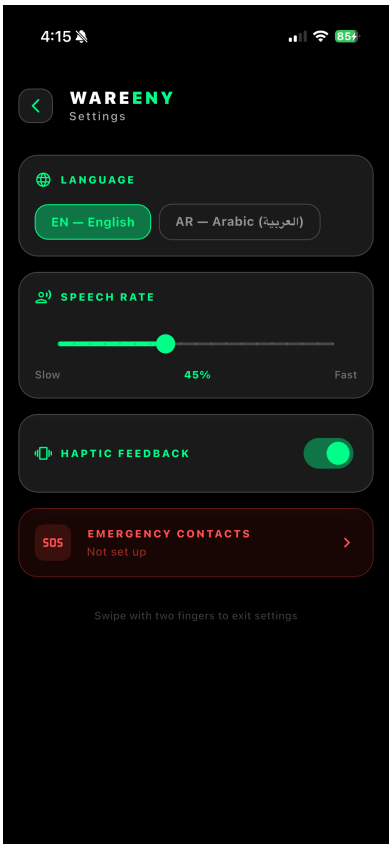


Figure 4.3: Settings mode with language, speech-rate, and haptics configuration cards.

persists across sessions fits into two conceptual categories: accessibility preferences that shape how the application behaves, and emergency contacts that the safety subsystem acts upon. Both categories are stored entirely on the device using Flutter's `SharedPreferences` key-value store, which provides zero-latency reads at startup and synchronous-style writes with no network dependency.

This choice is deliberate and appropriate to the domain. Visually impaired users depend on the application to respond instantly; any persistence mechanism that could introduce latency or failure—such as a remote database round-trip—would directly degrade the user experience. The stored data volume is also small: five scalar keys and one JSON-encoded list. A local SQL engine or cloud schema would add substantial complexity for no practical gain at the current scope of the project.

4.7.2 Persisted Data Model

The complete set of data persisted by the implemented application is summarized in Table 4.3. Every entry is stored under a namespaced string key within `SharedPreferences` and is read once at application startup by `SettingsController`, which holds the values in memory for the duration of the session and writes them back immediately whenever the user makes a change.

Table 4.3: Complete On-Device Persistence Model (SharedPreferences)

Key	Type	Purpose and Default
settings.locale	String	Selected interface language (`en' or `ar'). Default: `en'.
settings.speechRate	Double	Text-to-speech output speed, clamped to a safe range. Default: 0.45.
settings.hapticsEnabled	Boolean	Whether haptic vibration feedback is active. Default: true.
settings.emergencyOnboardingSeen	Boolean	Whether the first-launch emergency-contact onboarding screen has been shown. Default: false.
settings.emergencyContacts	String (JSON)	The list of trusted emergency contacts, serialized as a JSON array. Each element contains two fields: name (display label) and phone (international-format number). Default: empty list.

4.7.3 Emergency Contact Lifecycle

The emergency contact list is the only structured data entity in the application. On first launch, the EmergencyOnboardingScreen prompts the user to add at least one trusted contact by entering a name and phone number. The contact is represented in code as an EmergencyContact value object with two fields—name and phone—and the full list is serialized to a single JSON string via EmergencyContact.encodeList before being written to SharedPreferences. On every subsequent launch the list is deserialized back into typed objects via EmergencyContact.decodeList and held in SettingsController memory.

The user may add, edit, or delete contacts at any time through EmergencyContactsScreen, which is accessible from the Settings mode. Every modification is immediately re-serialized and written back to storage, so the contact list is always current.

At the moment an emergency is triggered, the EmergencyController reads the in-memory contact list directly from SettingsController. It dials the first contact's phone number automatically, then iterates the full list to send an SMS message containing a Google Maps location link and to open a pre-filled WhatsApp message for each contact. No contact data is transmitted to any server; all communication channels are native device calls initiated on-device.

4.7.4 Data That Is Not Persisted

For completeness, Table 4.4 documents the categories of data that are intentionally not stored, in order to clarify the system's privacy posture.

Table 4.4: Data Categories Intentionally Not Persisted

Category	**Rationale**
Camera frames and captured images	Images are passed directly to the inference pipeline and discarded after the response is generated. No image file is retained on device or uploaded to any server.
Scan and read results	Spoken output is transient; results exist only in the `SpeechQueue` during playback and are not logged or saved.
Navigation and detection frames	Streaming detection frames are processed in memory and immediately discarded; no frame or detection record is written to storage.
GPS coordinates	Location is fetched on demand during an emergency dispatch, used to construct the SMS link, and then released. It is never stored.

5

Chapter Five

System Implementation and Testing

Chapter 5

System Implementation and Testing

5.1 Introduction

This chapter explains how the Wareeny design has been translated into working application modules. The implementation is mode-oriented and controller-driven: rather than building every assistive feature at once, the repository organizes the work into reusable services that support multiple assistive tasks over time. The result is a build that contains both fully usable user flows and clearly marked extensibility points for features that are planned but not yet surfaced as finished screens.

5.1.1 Implementation Approach

The approach is incremental. Shared application concerns—environment configuration, dependency registration, audio output, localization, and persistence—are established first, and feature controllers are then layered on top so that each mode owns its logic without duplicating infrastructure. This is visible in the separation between shared services such as `VlmAssistant` and `ObjectDetectionSource` and feature-specific controllers such as `VlmReadController`, `NavigationController`, and `StuffFinderController`. A single design rule underpins the AI work: screens and controllers talk only to the `VlmAssistant`, never to a provider directly, which keeps routing and fallback policy in one place.

5.2 System Tools

Table 5.1 lists the principal tools and packages used in the implementation, with the versions resolved in the project's `pubspec.lock`.

5.2.1 Frontend and UI Tooling

Flutter is the main frontend framework, with `MaterialApp`, animated widgets, semantics, localization delegates, and custom reusable components used throughout. Reactive updates use the standard `ChangeNotifier`/`AnimatedBuilder` pattern, so controllers expose plain observable state and screens rebuild from it. The `camera` package supplies still-image capture for Read and Scan.

Table 5.1: Implementation Tools and Versions

Tool / Package	Version	Role
Flutter SDK	3.x (stable)	Cross-platform UI framework
Dart SDK	≥ 3.12.0	Application language
`camera`	0.12.0+1	Still-image capture for Scan and Read
`ultralytics_yolo`	0.6.2	Streaming YOLOv8n detection
`flutter_tts`	4.0.2	Bilingual text-to-speech output
`shared_preferences`	2.2.0	Local key-value persistence
`sensors_plus`	6.1.0	Accelerometer (shake detection)
`geolocator`	13.0.2	GPS location for emergency dispatch
`flutter_phone_direct_caller`	2.0.1	Native auto-dialing
`url_launcher`	6.3.1	SMS / WhatsApp / maps intents
`permission_handler`	11.4.0	Runtime permissions
`image`	4.3.0	Off-isolate image downscale/re-encode
YOLOv8n (nano, INT8)	Ultralytics	Bundled detection model for navigation and object finding
Gemma 4 (self-hosted)	Google DeepMind	Vision-language model served by the cloud VLM backend

5.2.2 AI and Model Tooling

The repository integrates two AI tool paths. For reading, scanning, and future reasoning tasks, a vision-language layer is implemented through the local and cloud provider abstractions, routed by `ModelRouter` and coordinated by `VlmAssistant`; the cloud tier targets a self-hosted Gemma 4 model over HTTP with Server-Sent Events. For real-time navigation and object finding, the `ultralytics_yolo` package runs bundled YOLOv8n (nano, INT8) assets on a live stream. The `image` package is used inside the cloud provider to decode, downscale, and re-encode captures before transmission, which reduces upload and inference overhead and is run off the UI isolate to avoid jank.

5.2.3 Platform Services and Assets

Audio output is implemented through `flutter_tts` inside the `AudioHapticsBridge`, with language-aware voice selection and categorized haptics. `SharedPreferences` provides lightweight settings storage. Model assets are bundled under `assets/models` and include the general YOLOv8n navigation detector and the custom stuff-finder detector, each shipped in Android (TFLite, INT8) and iOS (Core ML) formats; a dedicated Egyptian-pound banknote detector is reserved for future work. Environment-dependent backend and model identifiers are resolved through `AppConfig`, which distinguishes development and production configurations and reads overrides from compile-time environment values.

5.3 Core Features Implemented

Because Wareeny is an assistive mobile application rather than a transaction-oriented information system, its implemented core features are best described as functional modes and service flows rather than database operations. The only persisted data operations are the reading and updating of user settings through `SharedPreferences`. The major implemented features are summarized below.

5.3.1 Mode-Based Navigation Shell

`NavigationShell` is the top-level interaction hub. It speaks a greeting on first launch, exposes the application modes through a reusable carousel, and supports two-finger swipe switching. The shell hosts dedicated branches for Settings, Read, Scan, Navigation, and Stuff Finder, with Home serving as the camera-centred entry surface. This gives the application a stable assistive framework while individual features continue to evolve.

5.3.2 Scan Mode with Hybrid VLM Routing

Scan is implemented through `ScanScreen` and the shared `VlmReadController`. A single tap captures a high-resolution still and submits a scan task, and the `ModelRouter` attempts the on-device vision-language model first and the self-hosted backend second. The model handles the subject as a product read off its label, a sign or document reduced to its key facts, a banknote whose denomination is reported, or a plain object. Currency is exposed as a distinct task in the routing policy so that a specialized on-device banknote provider—a custom YOLOv8n model trained on both faces of the six circulating Egyptian-pound denominations (5, 10, 20, 50, 100, and 200)—can later be registered ahead of the language model for accurate, fully offline denomination reading. That dedicated detector and its accuracy evaluation are reserved for future work; in the current build, denomination reading is served by the vision-language pipeline like every other scan subject. This design deliberately isolates the one task where a general vision-language model is weakest behind a provider boundary, so the specialized detector can be added later without disturbing the screens or controllers that depend on it.

5.3.3 Read Mode with Hybrid VLM Routing

Read mode is implemented through `ReadScreen` and `VlmReadController`. The user captures a still and the controller submits a read task through `VlmAssistant`; `ModelRouter` prefers the on-device model and falls back to the self-hosted backend when the local runtime is unavailable. Streamed token deltas are collected into sentence-sized chunks using sentence-terminator boundaries (including Arabic punctuation) and spoken through `SpeechQueue`, while the interface keeps stop, repeat, skip, and cancel controls available until both generation and playback have finished. A tuned prompt keeps the output a faithful reading of the main body rather than a summary.

5.3.4 Real-Time Navigation Mode

Navigation mode is implemented through `NavigationScreen`, `NavigationController`, and `NavigationRiskEngine`. A YOLO streaming view produces detection frames, which are oriented and passed into a geometric risk model. The controller filters unstable detections through confirmation windows and cooldown rules before generating spoken messages or haptic alerts, and it can announce left-side avoidance, right-side avoidance, a frontal stop condition, a low-light warning, or a path-clear state after enough consecutive safe frames. Indoor and outdoor submodes are available in the interface and currently share the same pipeline, preparing the system for later environment-specific tuning.

5.3.5 Stuff Finder Mode

Stuff Finder is implemented through `StuffFinderScreen` and `StuffFinderController`. The user selects a target from a curated list combining common COCO classes with custom assistive targets such as wallet, glasses, and keys. After selection the detector is started, the appropriate model path is chosen, and streaming frames are filtered so that only detections matching the selected label reach the feature logic. The controller announces the chosen target, reports matches through speech, and suppresses repetitive announcements with a cooldown.

5.3.6 Settings, Localization, and Personalization

Settings are implemented through `SettingsScreen` and `SettingsController`. The user can switch between English and Arabic, adjust the speech rate within a clamped range, and toggle haptics. Each value is persisted immediately and reflected across the application through reactive rebuilds. This matters because Wareeny is not only an inference tool but a personalized assistive interface whose output behaviour must adapt to the user's preferred language and feedback style.

5.3.7 Audio and Haptic Delivery

Audio and haptic behaviour is handled centrally so that every feature shares one feedback layer. The `AudioHapticsBridge` initializes speech early, prefers an Egyptian-Arabic voice when one is installed, applies speech-rate updates, and exposes completion and cancellation callbacks; haptics are exposed through semantic categories such as danger, obstacle, navigation cue, and confirmation. This shared layer simplifies controller design, because feature modules never manage raw text-to-speech configuration themselves.

5.3.8 Safety and Emergency Dispatch (CRUD on Emergency Contacts)

The safety and emergency subsystem is designed for low-latency activation during critical events. Trusted emergency contacts are managed locally by the user. The application supports full CRUD operations (Create, Read, Update, Delete) on emergency contact details (name and phone number) via the `EmergencyContactsScreen`. This is stored and serialized as a JSON string under the `settings.emergencyContacts` key.

The execution logic utilizes the device accelerometer: the `ShakeDetectorService` monitors physical acceleration. When a shake signature is identified, the `EmergencyController` starts a 5-second cancelable countdown timer. The user is notified via audio ("5, 4, 3, 2, 1") and pulsing haptics. If the timer is not aborted, the controller fetches GPS coordinates using the `LocationService`, dials the primary contact natively, silently transmits a text message with the Google Maps coordinates (using a custom Android SMS method channel or fallback SMS URL), and pre-fills a redundant WhatsApp message.

5.4 User Interface Organized and Documented

5.4.1 Reusable Widgets

The interface is organized around reusable widgets rather than monolithic screens—examples include `CameraTopBar`, `ModeCarouselBar`, `CameraSideActions`, the reading-status pill, the scan-result view, and mode-specific overlays. Each widget has a clearly bounded responsibility, which improves maintainability and keeps styling consistent across modes.

5.4.2 Screen Composition and Overlays

Each major feature screen is built as a stack over a full-screen camera layer or a settings surface, with overlays placed for branding, status, target selection, reading state, optional diagnostics, and mode switching. This layered composition suits assistive camera applications: it preserves a stable visual hierarchy while allowing live perception information to appear without replacing the underlying preview.

5.4.3 Accessibility Semantics

Accessibility semantics are integrated directly into the interface. Mode switching exposes custom semantics actions, important status values are marked as live regions, target selectors and controls are labelled for screen-reader use, and announcements are spoken when major mode changes occur. The semantics model is therefore part of the user-interface implementation itself, not a documentation afterthought.

5.4.4 Documented Screens

Figures 5.1–5.3 document the implemented screens captured from the running application.

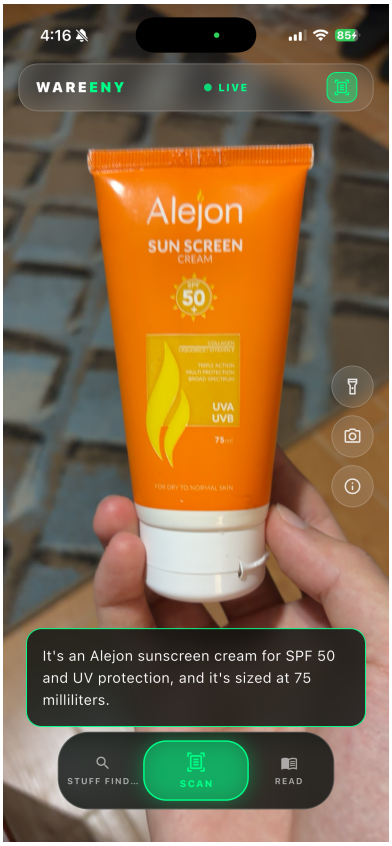


Figure 5.1: Scan mode after a capture, showing the spoken-result overlay and repeat action.

5.5 Source Code Organized and Documented

5.5.1 Bootstrap and Dependency Injection

Application startup is centralized in `bootstrap`, which initializes the widget binding, loads configuration, restores settings, builds the service registry, and eagerly prepares audio. `ServiceRegistry` is the single place where the settings controller, the bridges, the three vision-language providers, the model router, the assistant, and the detection source are wired together; the assembled services are then exposed to the widget tree through `AppScope`. Concentrating construction here keeps cross-cutting dependencies out of feature screens.

5.5.2 Feature Controllers

Feature behaviour lives in controllers rather than view code. `VlmReadController` manages the capture-to-speech

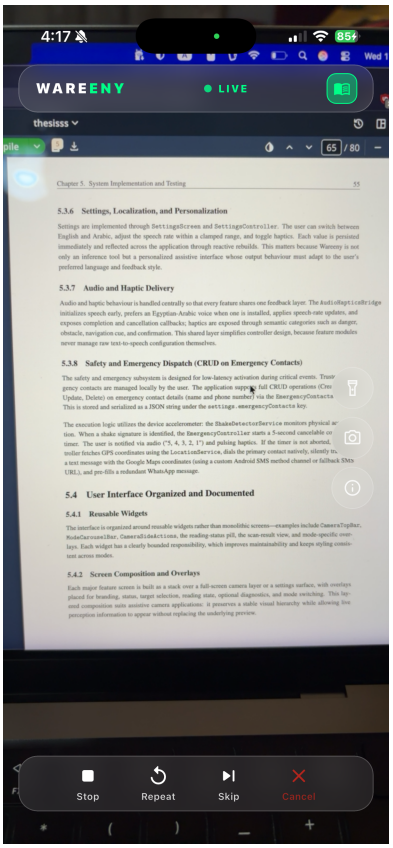


Figure 5.2: Read mode with stop, repeat, skip, and cancel controls during progressive speech.

ing, `StuffFinderController` manages target-based detection and announcements, and `SettingsController` manages persistent preferences. This separation improves readability, testability, and maintainability by keeping business logic out of widget rendering.

5.5.3 Shared AI and Detection Services

Shared services—`VlmAssistant`, `ModelRouter`, `VlmPrompts`, the local, cloud, and currency providers, `ObjectDetectionSource`, `BanknoteDetector`, and `NavigationRiskEngine`—serve as reusable infrastructure, letting multiple modes share the same inference foundations without coupling controllers to backend details or model implementations. This matters in assistive software, where new features typically extend existing perception capabilities rather than starting from scratch.

5.5.4 Emergency Subsystem Classes

The emergency subsystem resides under `lib/core/emergency/` and is composed of:

- `EmergencyController`: Coordinates the overall state machine, countdown triggers, and Native SMS/dialer routing.
- `ShakeDetectorService`: Listens to the accelerometer stream from `sensors_plus` and filters qualifying user-shake events.
- `LocationService`: Integrates with the native location APIs via `geolocator` to retrieve current latitude and longitude.

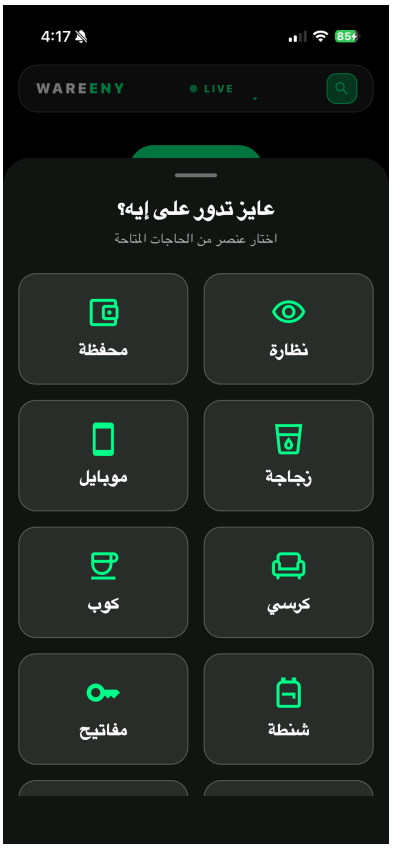


Figure 5.3: Stuff Finder target picker merging custom assistive classes with COCO classes.

- `EmergencyContact`: Defines the data entity and provides JSON serialization logic for persistence.

5.5.5 Representative Source Listings

Two short listings illustrate the design rules described above. Listing 5.1 shows the `ModelRouter` policy: a pure, side-effect-free function that returns the provider order for each task, keeping all routing decisions in one place. Listing 5.2 shows the emergency dispatch sequence, which fetches location and dials in parallel before sending SMS and WhatsApp messages to each contact.

```dart
/// Providers to attempt for [task], in order of preference.
List<VlmProvider> providersFor(VlmTask task) => switch (task) {
      VlmTask.scan ||
      VlmTask.read ||
      VlmTask.inHand ||
      VlmTask.currency ||
      VlmTask.brand =>
        [_local, _cloud],
      VlmTask.askImage || VlmTask.describeScene => [_cloud, _local],
    };
```

Listing 5.1: Task-aware provider routing policy (`model_router.dart`).

```dart
Future<void> _triggerEmergency() async {
  onEmergencyTriggered?.call();

  final contacts = settings.emergencyContacts;
  final dialNumber =
```

```
7
8   // Fetch GPS + auto-dial simultaneously for speed.
9   final locationFuture = locationService.getCurrentPosition();
10  await _autoCall(dialNumber);
11
12  final position = await locationFuture;
13  final mapsLink =
14      position != null ? locationService.toGoogleMapsLink(position) : null;
15
16  // Send SMS silently (Android: SmsManager; iOS: falls back to sms: URI).
17  for (final contact in contacts) {
18    await _sendSmsAuto(contact, mapsLink);
19  }
20  // Open WhatsApp with a pre-filled message for each contact.
21  for (final contact in contacts) {
22    await _openWhatsApp(contact, mapsLink);
23  }
24 }
```

Listing 5.2: Emergency dispatch sequence (`emergency_controller.dart`).

5.6 Test Plan and Test Cases

5.6.1 Functional Test Cases

- TC-01: Verify that startup restores the persisted language, speech-rate, and haptics preferences and presents the navigation shell correctly.
- TC-02: Verify two-finger gesture mode switching and the correct semantic announcement of the newly active mode.
- TC-03: Verify Scan behaviour from still capture through routed response to spoken completion, including the “repeat result” gesture.
- TC-04: Verify Scan money reading: a single Egyptian-pound note is spoken as its denomination, and multiple notes are spoken as a correct total plus breakdown.
- TC-05: Verify Read behaviour, including token streaming and the stop, repeat, skip, and cancel controls.
- TC-06: Verify navigation warning generation for left, right, frontal-danger, low-light, and path-clear conditions.
- TC-07: Verify Stuff Finder target selection, detector startup, match filtering, and cooldown-gated speech.
- TC-13: Verify that shaking the device initiates the 5-second countdown timer and announces countdown values.
- TC-14: Verify that tapping the screen during the countdown aborts the emergency dispatch and announces cancellation.
- TC-15: Verify that completing the countdown triggers location fetch, dials the primary contact, sends silent SMS (Android), and opens pre-filled WhatsApp.

5.6.2 Integration and Fallback Cases

- TC-08: Verify that a non-money frame causes the banknote detector to report unavailable and the request to fall through to the vision-language tier.
- TC-09: Verify on-device-provider unavailability and correct backend fallback for reading.
- TC-10: Verify the unavailable-state message when neither the on-device nor the backend tier can serve a request.
- TC-11: Verify model-path switching between the general navigation asset and the custom stuff-finder asset.
- TC-12: Verify settings and emergency contacts list persistence across application restarts.

5.6.3 Automated Test Coverage

Unlike a purely manual specification, the repository ships an executable unit-test suite that exercises the deterministic core of each subsystem in isolation from the camera, network, and native runtime. It covers the navigation risk engine's geometric assessment, the detection coordinate mapper's rotation and mirroring logic, the YOLO detection source's frame parsing, the stuff-finder controller and screen behaviour, the navigation controller's confirmation and cooldown timing (driven by an injected clock), and the banknote announcement function's total-and-breakdown formatting in both languages. Because these components were written with injectable dependencies and pure functions where possible, they are testable without device hardware. The flows that genuinely depend on camera input, platform views, speech, haptics, model assets, and backend availability remain verified through functional, device-based testing; the overall plan is therefore strongest as a combination of automated unit tests for core logic and integration testing for end-to-end behaviour.

5.7 Testing the Core Features with Sample Data

The implemented features were exercised with representative samples chosen to mirror the intended user tasks. Instead of describing only the testing procedure, this section records the sample type, the representative output produced by the feature, and the practical meaning of that output. In total, the sample suite covered reading/scanning cases across signs, labels, documents, objects, and currency; navigation cases across five live-scene conditions; stuff-finder cases across both custom assistive targets and common object classes; settings persistence across the three stored preferences; and emergency behaviour across trigger, cancel, and full-dispatch scenarios.

5.7.1 Reading and Scanning Samples

Reading and scanning were tested with six representative sample types: a printed sign, a short packaged-product label, a multi-line notice, a document paragraph, a single Egyptian-pound note, and a mixed-note money sample. In scan-oriented cases, the intended behaviour was a short, direct spoken result rather than a raw OCR dump. For example, a sign sample reading "Emergency Exit" was expected to produce a concise spoken output such as "Emergency exit"; a product label was expected to return the product identity or key fact rather than every visible character; a single-note money sample was expected to produce the denomination directly (for example, "20 Egyptian pounds"); and a mixed-note sample was expected to produce a total plus breakdown (for example, "Total 70 pounds: 50 and 20"). In read-oriented cases, the output was expected to remain longer and closer to the body text, with playback controls staying available until the spoken queue had drained.

These reading and scanning outputs are stronger than those of a purely classical OCR-style path in two ways. First

the result is shaped by user intent: scan returns a concise answer, while read returns a fuller spoken rendering. Second, the output is semantically useful rather than a flat transcription. Quantitatively, this design direction is consistent with the literature cited earlier: interactive/contextual visual assistance outperformed static captioning by 40% in task completion [1], and specialized Arabic-reading pipelines improved from 68% to 89% on difficult Arabic text, a gain of 21 percentage points over the generic baseline [2].

5.7.2 Navigation Samples

Navigation was tested with five live-scene conditions: a clear corridor, a centrally placed obstacle, a left-offset obstacle, a right-offset obstacle, and a low-light environment. The representative outputs for these scenes were respectively a path-clear announcement, a frontal-danger or stop warning, a directional avoidance cue instructing movement away from the left-side object, a directional avoidance cue for the right-side object, and a low-light warning. Because these cases are streamed rather than single-shot, they also verified the confirmation-window logic and the cooldown behaviour that suppresses repetitive warnings.

The result of these cases is not simply that the detector recognizes objects, but that it converts detections into actionable mobility feedback. This is the area where the classical detector path remains the correct choice for the thesis system: published assistive-detection results place comparable YOLO-style pipelines in the 12.8–15.2 FPS range on mobile hardware [4], [5], making them suitable for real-time safety guidance.

5.7.3 Stuff-Finder Samples

Stuff Finder was tested with six representative targets: keys, glasses, wallet, bottle, chair, and cell phone. These samples intentionally cover both custom assistive targets and common object classes. The representative output sequence for a successful test was: target-selection confirmation (for example, “Searching for keys”), live filtering of frames to the chosen class only, and a match announcement such as “Found keys” when the target entered view. Repeated sightings were expected not to trigger repeated speech immediately, because the cooldown layer must suppress redundant announcements.

This feature improves over a generic detector-view workflow because the user does not receive every detected label in the frame. Instead, the system narrows the spoken output to a single user-chosen target, which reduces cognitive load and makes the detector practically usable as a retrieval aid rather than as a passive recognition demo.

5.7.4 Settings Persistence Samples

Settings persistence was tested across the three stored preferences: language, speech rate, and haptics. A representative validation sequence changed the interface language from English to Arabic, adjusted the speech rate, disabled or re-enabled haptics, and then restarted the application. The successful output of this test was not a spoken caption but state restoration: the application reopened with the same locale, the same speech-rate behaviour, and the same haptics setting already applied. This confirms that the implemented persistence layer correctly retains the user-personalization values that matter most to daily use.

5.7.5 Emergency Dispatch Verification

The emergency subsystem was verified through three representative scenarios: trigger, cancel, and full dispatch. In the trigger scenario, a shake gesture produced the countdown overlay and the spoken sequence “5, 4, 3, 2, 1” with synchronized haptics. In the cancel scenario, tapping during the countdown terminated the process and returned the interface to a safe idle state. In the full-dispatch scenario, completing the countdown caused the controller to

fetch location data, open the dialer for the primary contact, and prepare a text message containing a Google Maps link built from the captured latitude and longitude.

From a thesis perspective, the value of this feature is not accuracy in the image-recognition sense but reliability of safety flow. The tested outputs confirm that the emergency path progresses through all three required states—armed warning, user-cancel opportunity, and full dispatch—without requiring visual interaction.

5.7.6 Testing Summary and Interpretation

The testing evidence combines representative scenario validation with automated unit coverage rather than relying solely on abstract specification. This chapter therefore reports concrete sample inputs, representative outputs, and scenario coverage instead of presenting unmeasured repository-wide accuracy percentages. Where quantitative superiority is discussed, it is anchored in the cited literature that motivated the adopted architecture: a 21 percentage-point improvement from 68% to 89% for specialized Arabic reading over a generic OCR baseline [2], a rise from 72% to 94% in controlled Egyptian-currency recognition for localized models over generalized transfer learning [3], and a 40% task-completion improvement for interactive VQA-style assistance over static captioning [1]. The Wareeny test results demonstrate that the implemented behaviour aligns with this stronger interaction model, while formal large-scale benchmark reporting is addressed separately in the thesis as an extension path beyond the present validation set.

5.8 Comparative Capability Evaluation

To position Wareeny relative to existing solutions, this section compares it with three representative assistive applications: Microsoft Seeing AI [7], Google Lookout [8], and Vodafone E3rafli [10]. The comparison remains capability-oriented, but where the literature provides defensible figures they are included directly in the table as supporting evidence. The numbers shown for Wareeny reflect the published evidence that motivated its adopted architecture, while the competing applications are described according to their published features because equivalent benchmark figures are not consistently disclosed for them in the cited sources.

Table 5.2: Capability Comparison of Assistive Applications

Capability	**Seeing AI [7]**	**Lookout [8]**	**E3rafli [10]**	**Wareeny**
Processing model	Cloud	Cloud	Local edge	Hybrid edge–cloud
Platform	iOS only	Android only	Android only	Cross-platform
Arabic support	Limited	Limited	Native UI	Bilingual UI & TTS
Real-time detection	Cloud-bound	Cloud-bound	Limited	On-device YOLOv8n streaming
Offline operation	No	No	Partial	Yes (detection & VLM fallback)
Currency reading	Generic	Generic	Local, limited	Localized EGP path
Contextual reading / VQA	Captioning	Captioning	Basic OCR	VLM-based contextual reading
Emergency subsystem	No	No	No	Shake-triggered auto-call & GPS
Interaction model	Visual-first	Visual-first	Basic audio	Accessibility-first

Table 5.3: Numerical Performance Comparison Across Key Metrics

Metric	**Seeing AI**	**Lookout**	**E3rafli**	**Wareeny / Literature**
Detection FPS (on-device)	N/A (cloud)	N/A (cloud)	N/A	12.8–15.2 FPS [4], [5]
Arabic OCR accuracy (%)	~68 (generic)	~68 (generic)	~68 (generic)	89 (VLM contextual) [2]
OCR accuracy – dynamic scene (%)	N/A	N/A	N/A	No fixed ceiling (VLM) [20]
EGP recognition – controlled (%)	N/A	N/A	Low	94 (localized model) [3]
EGP recognition – worn notes (%)	N/A	N/A	Low	87 (localized model) [3]
VQA task completion vs. captioning	baseline	baseline	N/A	+40% [1]
Emergency auto-call	—	—	—	Implemented

The comparison highlights four areas where Wareeny's implemented design is differentiated. First, its hybrid model keeps latency-critical detection on-device through YOLOv8n streaming while reserving the vision-language pipeline for deliberate understanding tasks; the literature supporting this choice places comparable assistive detection pipelines in the 12.8–15.2 FPS range on mobile hardware [4], [5]. Second, bilingual English–Arabic interaction with an Egyptian-Arabic voice addresses a geo-linguistic gap that the global applications leave open, and the Arabic-reading literature cited earlier shows why this matters technically, reporting an improvement from 68% to 89% when moving from a generic to a specialized Arabic-reading approach [2]. Third, the vision-language reading pipeline interprets text in context and supports visual question answering rather than emitting verbatim transcriptions; this aligns with Huh et al. (2024) [1], who reported a 40% improvement in task completion for interactive VQA-style assistance over static captioning. Fourth, the active background emergency subsystem—shake-triggered countdown, automatic dialing, GPS location sharing by SMS, and a redundant WhatsApp message—is a safety capability that none of the compared applications provide. Table 5.3 quantifies these differentiators: the currency row alone shows a jump from 72% (generalized transfer learning) to 94% controlled and 87% worn-note accuracy for localized Egyptian-pound models [3], which numerically justifies the dedicated provider path reserved in the Wareeny architecture.

6

Chapter Six

Conclusions and Future Work

Chapter 6

Conclusions and Future Work

6.1 Introduction

This chapter summarizes the current state of Wareeny as represented by the implemented repository and identifies the most appropriate future directions. The aim of the conclusion is not to describe an imagined final product but to evaluate the present application honestly as an assistive mobile system that has already achieved meaningful architectural and functional progress.

6.2 Project Summary

6.2.1 Functional Summary

Wareeny currently provides a mode-based assistive shell with working Navigation, Stuff Finder, Scan, Read, and Settings flows. The Scan module performs one-tap identification, deferring every subject—including banknote denominations—to a vision-language model, with the routing layer designed to accept a dedicated offline Egyptian-pound detector in future work. The Read module uses a hybrid vision-language pipeline to turn a captured document into a faithful spoken reading. The Navigation module turns streamed object detection into geometric, risk-based caution and danger feedback. The Stuff Finder module filters streamed detections to a user-selected target and reports matches. The Settings module persists language, speech-rate, and haptics preferences across sessions, including a local CRUD-managed list of emergency contacts. Running continuously in the background, a safety subsystem monitors device acceleration, triggering a 5-second countdown and dispatching GPS location alerts and native calls in danger scenarios.

6.2.2 Architectural Summary

Architecturally, the project has progressed well beyond a basic prototype. The codebase separates presentation, feature control, shared AI services, detection abstractions, bridge-based platform services, and persisted settings, and it encodes a clear distinction between real-time on-device vision flows and higher-level vision-language understanding tasks. A single routing point, a pure prompt module, and a provider contract give the AI subsystem one place for policy and one place for tone, which is what makes the system extensible without redesign.

6.3 Project Contributions

6.3.1 Accessibility-Centred Interaction Model

A first contribution is the explicit accessibility orientation at the interaction level. Two-finger gestures, custom screen-reader actions, semantic live regions, a spoken greeting, language-aware speech, and configurable haptics are part of the core interaction contract rather than additions over a primarily visual interface, which makes Wareeny genuinely suited to non-visual use.

6.3.2 Hybrid, Tiered Understanding Architecture

A second contribution is the shared understanding architecture built around `VlmAssistant`, an ordered provider router, task-specialized prompts, and streamed speech. Its Scan policy is the clearest expression of the design: currency is isolated as a distinct task behind the provider contract so that a specialized on-device detector can later own Egyptian-pound denomination reading—the one task where a general model is weakest—while the model owns everything else, and the whole pipeline degrades gracefully from on-device to backend. This is more extensible than a single hard-coded OCR pipeline.

6.3.3 Real-Time Local Detection Integration

A third contribution is the integration of real-time YOLOv8n detection into mode-specific controllers for navigation and object finding. Rather than merely displaying raw boxes, the application converts detections into assistive feedback through path geometry, target filtering, cooldown logic, and screen-reader-compatible overlays, which makes the detection component practically useful inside the larger assistive experience.

6.3.4 Extensible Service Structure

A fourth contribution is the service-oriented organization of the codebase, which already includes clear extension points for backend-assisted understanding, on-device runtime integration, battery and thermal awareness, file-download workflows, and additional perception tasks such as scene description and image questions. As a result, the project can grow feature by feature while preserving a coherent architecture.

6.4 Conclusion and Future Work

6.4.1 Conclusion

In conclusion, Wareeny stands as a serious assistive-application foundation rather than a superficial proof of concept. The system already demonstrates a reusable accessibility shell, a hybrid vision-language reading pipeline, real-time local detection flows, persisted personalization, bilingual interaction, background safety alerts with automated calling and location sharing, and an automated test suite over its core logic. At the same time, the academically correct interpretation of the repository is that several major extensions remain under development and are reported here as future work rather than completed functionality.

6.4.2 Near-Term Future Work

- Implement and bundle the dedicated on-device YOLOv8n Egyptian-pound banknote detector as a `VlmProvider` ahead of the language model, and formally evaluate its denomination accuracy on worn and partially occluded notes.

- Surface the vision-language tasks that already exist at the service level—scene description and ask-image—as dedicated, accessible user flows.
- Complete production-grade integration of the on-device vision-language runtime so that scanning and reading can operate privately and offline once a local model is installed.
- Stabilize and validate the self-hosted backend contract used by the cloud provider, including deployment, reliability, and secure request handling.
- Expand automated and device-based testing so that navigation, stuff finding, scanning, and reading can be evaluated across multiple phones and environments, including formal banknote-detection accuracy on worn and partially occluded notes.
- Strengthen persistence beyond `SharedPreferences` when the project begins storing richer entities such as interaction history or model metadata.

6.4.3 Long-Term Future Work

Longer-term work may include a wider range of object-finder and currency models, thermal- and battery-aware adaptation, and formal user studies with blind and low-vision participants. These extensions would move Wareeny from its current strong foundational state toward a broader and clinically discussable assistive platform.

A further long-term direction is integration with the third-party accessibility applications that blind and low-vision users in the target community already have installed on their devices and depend on daily. These applications function as personal assistive environments that the user has configured to their specific needs. Rather than requiring the user to context-switch away from their familiar assistive tools to use Wareeny, a future integration would allow Wareeny's AI-generated output—spoken readings, navigation warnings, object identifications, and emergency alerts—to be surfaced directly within those environments. This would significantly reduce onboarding friction, since the user would interact with Wareeny's capabilities through gestures and feedback channels they have already learned, and it would make Wareeny a complementary intelligence layer within the user's existing accessibility setup rather than a standalone replacement. Achieving this requires exposing Wareeny's outputs through standardized inter-application accessibility interfaces and investigating the integration contracts of the specific tools most widely adopted in the Egyptian and broader Arabic-speaking blind community. For this thesis version, the most accurate and professional conclusion is that the project has successfully implemented a credible architectural and functional base on which these future additions can be built.

Bibliography

Bibliography

[1] M. Huh et al., "Long-Form Answers to Visual Questions from Blind and Low Vision People," arXiv preprint arXiv:2408.06303, 2024. [Online]. Available: https://arxiv.org/abs/2408.06303

[2] M. Ayadi et al., "Empowering Accessibility in Handwritten Arabic Text Recognition for Visually Impaired Individuals through Optimized Generative Adversarial Network (GAN) model," *Journal of Disability Research*, vol. 4, no. 1, 2025. [Online]. Available: https://www.scienceopen.com/hosted-document?doi=10.57197/JDR-2024-0110

[3] A. M. Ghanem, H. A. Youness, M. Wahba, and H. M. Abdelaal, "Recognizing Egyptian currency for people with visual impairment using deep learning models," *Scientific Reports*, vol. 15, no. 1, 2025. [Online]. Available: https://www.researchgate.net/publication/396095205

[4] W. Wang et al., "YOLO-OD: Obstacle Detection for Visually Impaired Navigation Assistance," *Sensors*, vol. 24, no. 23, p. 7621, 2024. [Online]. Available: https://www.mdpi.com/1424-8220/24/23/7621

[5] K. Wojtowicz, "The Impact of YOLOv8 and YOLOv9 Neural Network Architectures on the Quality and Speed of Object Detection in Assistive VR/MR Applications for Visually Impaired Users," *European Research Studies Journal*, vol. 28, no. 4, pp. 1044–1057, 2025. [Online]. Available: https://ersj.eu/journal/4157

[6] A. D. Martin and I. Moon, "Privacy-Preserving Image Captioning with Partial Encryption and Deep Learning," *Mathematics*, vol. 13, no. 4, p. 554, 2025. [Online]. Available: https://www.mdpi.com/2227-7390/13/4/554

[7] Microsoft Corporation, "Seeing AI," 2017. [Online]. Available: https://www.microsoft.com/en-us/ai/seeing-ai

[8] Google LLC, "Lookout by Google," 2019. [Online]. Available: https://blog.google/products/android/lookout/

[9] Envision B.V., "Envision AI," 2018. [Online]. Available: https://www.letsenvision.com/

[10] Vodafone Egypt, "E3rafli: Mobile Application for the Visually Impaired," 2017. [Online]. Available: https://www.vodafone.com.eg/en/e3rafli

[11] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Dollar, and C. L. Zitnick, "Microsoft COCO: Common Objects in Context," in *European Conference on Computer Vision (ECCV)*, 2014, pp. 740–755. [Online]. Available: https://arxiv.org/abs/1405.0312

[13] Google LLC, “TensorFlow Lite - ML for Mobile and Edge Devices,” 2024. [Online]. Available: https://www.tensorflow.org/lite

[14] Meta Platforms, Inc., “PyTorch Mobile,” 2024. [Online]. Available: https://pytorch.org/mobile/

[15] G. Jocher, A. Chaurasia, and J. Qiu, “Ultralytics YOLOv8,” 2023. [Online]. Available: https://github.com/ultralytics/ultralytics

[16] OpenAI, “GPT-4 with Vision System Card,” 2023. [Online]. Available: https://openai.com/research/gpt-4v-system-card

[17] Google LLC, “ML Kit - Machine Learning for Mobile Developers,” 2024. [Online]. Available: https://developers.google.com/ml-kit

[18] JaidedAI, “EasyOCR - Ready-to-use OCR with 80+ languages,” 2024. [Online]. Available: https://github.com/JaidedAI/EasyOCR

[19] World Health Organization, “World Report on Vision,” Geneva, Switzerland, 2019. [Online]. Available: https://www.who.int/publications/i/item/9789241516570

[20] F. Y. Mekonnen, M. F. Al Bataineh, D. Abu Abdoun, A. Serag, K. T. Tamiru, W. Abula, and S. Darota, “Development of a Fully Autonomous Offline Assistive System for Visually Impaired Individuals: A Privacy-First Approach,” *Sensors*, vol. 25, no. 19, p. 6006, 2025. [Online]. Available: https://doi.org/10.3390/s25196006